Software Engineering
AIML 206

Semester: IV

Overview of Course, Basics of SE

Learning objectives:

The paper aims to understand the importance, limitations and challenges of processes involved in software development. In this course, the learners will be able to develop expertise related to the following:

1. To gain knowledge of various software models.
2. To gain knowledge of various software design activities.
3. To learn cost estimation, software testing, Maintenance and debugging

COURSE OUTCOMES (COs):

CO #	Detailed Statement of the CO
CO1	Understand software systems of the real world and their life cycle.
CO2	Design the software solutions per the SRS requirement and proper tools.
CO3	Estimate software development cost and its maintenance.
CO4	CO4 Deploy various testing techniques to test software.

Syllabus

UNIT – I

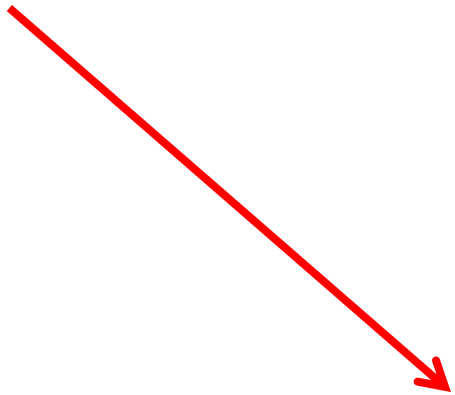
Introduction to Software- Nature of Software, Introduction to Software Engineering, Software Engineering Layers, Software Myths, The Software Processes, Project, Product, Process Models: A Generic Process Model, Waterfall Model, Incremental Process Models, Evolutionary Process Models, Spiral Model. COCOMO Model. UML diagrams -Sequential, Class Diagram, Activity Diagram, Component Diagram, Use-Case Diagram, State Machine Diagram.

Books

- **TEXT BOOKS:**
- **TB1. K. K. Aggarwal & Yogesh Singh, "Software Engineering", 2nd Ed., New Age International, 2005.**
- **TB2. I. Sommerville, "Software Engineering", 9th Edition, Pearson Edu.**
- **REFERENCE BOOKS:**
- **RB1. Jibitesh Mishra and Ashok Mohanty, "Software Engineering", Pearson**
- **RB2. R. S. Pressman, "Software Engineering – A practitioner's approach", 5th Ed., McGraw Hill RB3. James Peter, W. Pedrycz, "Software Engineering: An Engineering Approach", John Wiley & Sons**

Why Software Engineering ?

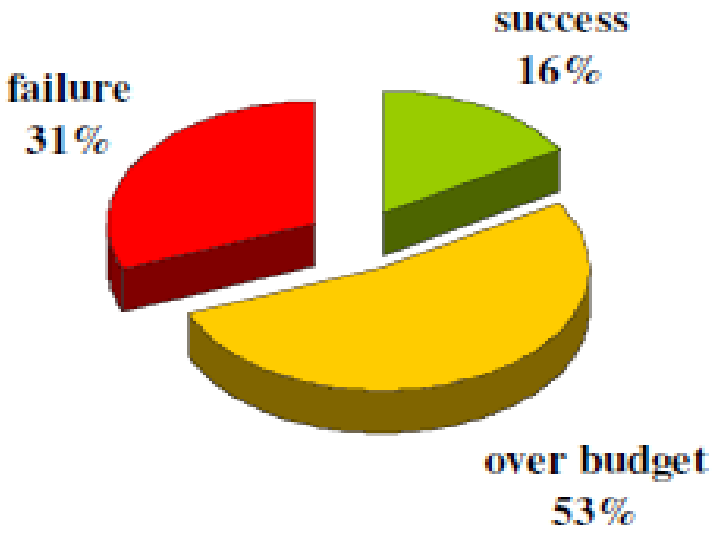
- Change in nature & complexity of software
- Concept of one "guru" is over
- We all want improvement



- Ready for change

The Evolving Role of Software

Software industry is in Crisis!



Software Crisis

- Software industry is in Crisis

- As per the IBM report, “31%of the project get cancelled before they are completed, 53% over- run their cost estimates by an average of 189% and for every 100 projects, there are 94 restarts

Some software failures

- **Ariane 5**: **I**t took the European Space Agency **10 years and $7 billion** to produce Ariane 5, a giant rocket capable of hurling a pair of three-ton satellites into orbit with each launch and intended to give Europe overwhelming supremacy in the commercial space business.
- The rocket was destroyed after 39 seconds of its launch, at an altitude of two and a half miles along with its payload of four expensive and uninsured scientific satellites.

Ariane 5:

- When the guidance system's own computer tried to convert one piece of data the sideways velocity of the rocket from a 64 bit format to a 16 bit format; the number was too big, and an overflow error resulted after 36.7 seconds. When the guidance system shutdown, it passed control to an identical, redundant unit, which was there to provide backup in case of just such a failure. Unfortunately, the second unit, which had failed in the identical manner a few milliseconds before.

Y2K Problem:

- It was simply the ignorance about the adequacy or otherwise of using only last two digits of the year.
- The 4-digit date format, like 1964, was shortened to 2-digit format, like 64.
- As the year 2000 loomed, there was concern that systems would interpret the year "00" as 1900 instead of 2000. This could cause errors in calculations, data processing, or system failures, particularly in critical infrastructure like banking, utilities, transportation, and healthcare.

The Patriot Missile

- First time used in Gulf war
- Used as a defense from Iraqi Scud missiles
- Failed several times including one that killed 28 US soldiers in Dhahran, Saudi Arabia
- **Reasons:**
- A small timing error in the system's clock accumulated to the point that after 14 hours, the tracking system was no longer accurate. In the Dhahran attack, the system had been operating for more than 100 hours.

Financial Software:

- Many companies have experienced failures in their accounting system due to faults in the software itself. The failures range from producing the wrong information to the whole system crashing.

No silver bullet

- *Silver Bullet: It* argues that there is no single, magical solution or breakthrough that will solve all the inherent challenges of software development.
- The hardware cost continues to decline drastically.
- However, there are desperate cries for a silver bullet something to make software costs drop as rapidly as computer hardware costs do.
- But as we look to the horizon of a decade, we see no silver bullet. There is no single development, either in technology or in management technique, that by itself promises even one order of magnitude improvement in productivity, in reliability and in simplicity

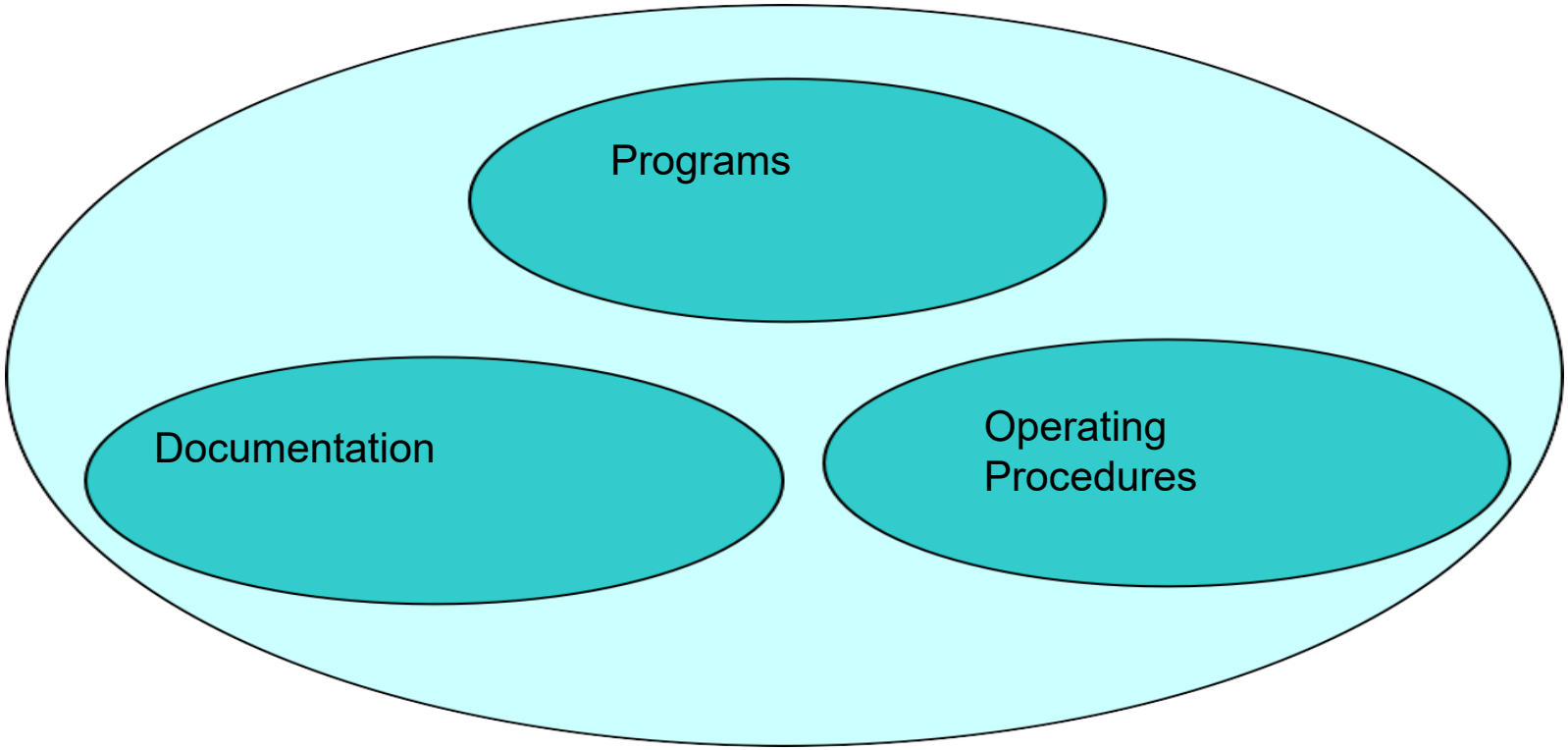
No silver bullet

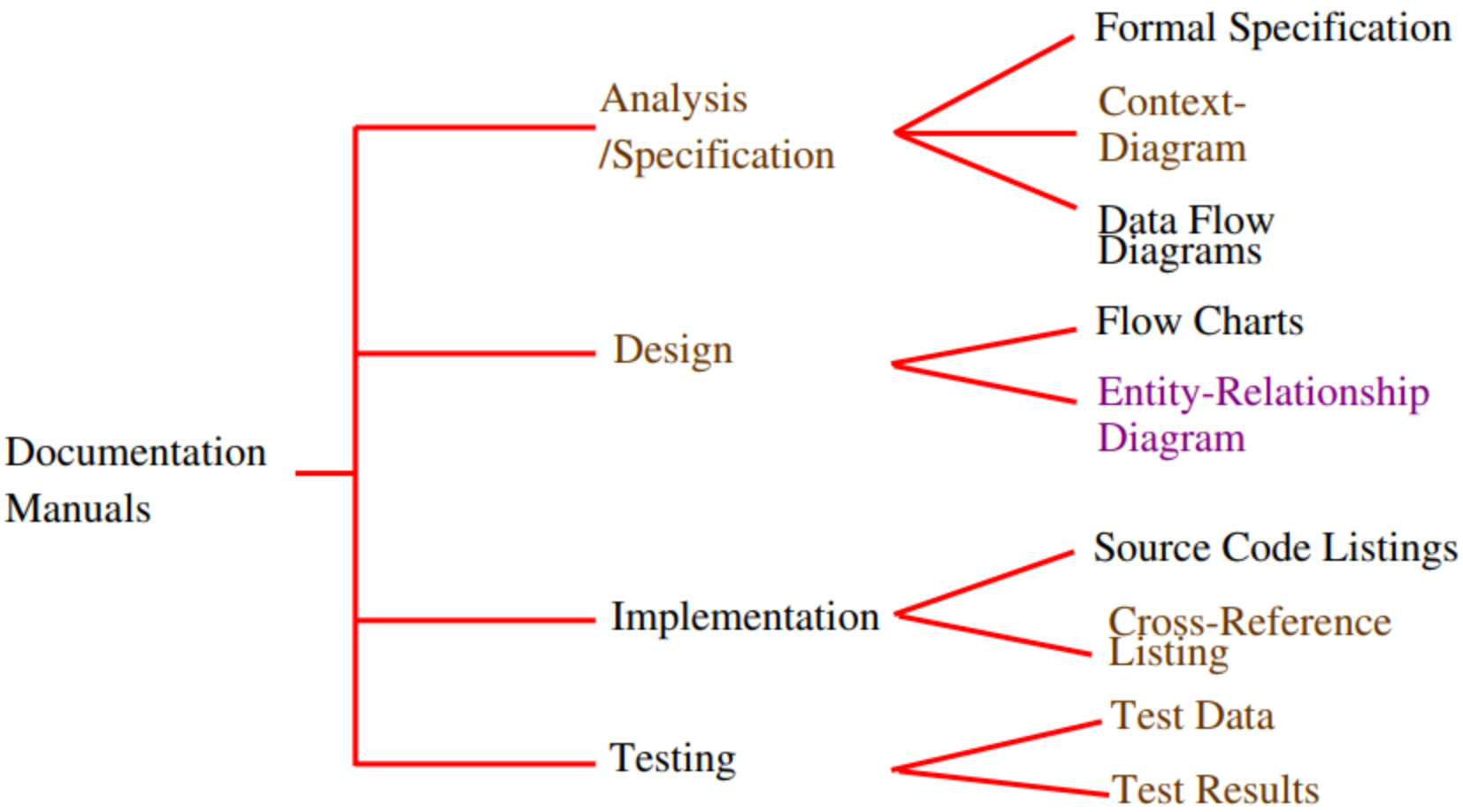
- The hard part of building software is the specification, design and testing of this conceptual construct, not the labour of representing it and testing the correctness of representation.
- We still make syntax errors, to be sure, but they are trivial as compared to the conceptual errors (logic errors) in most systems. That is why, building software is always hard and there is inherently no silver bullet.

What is a Software?

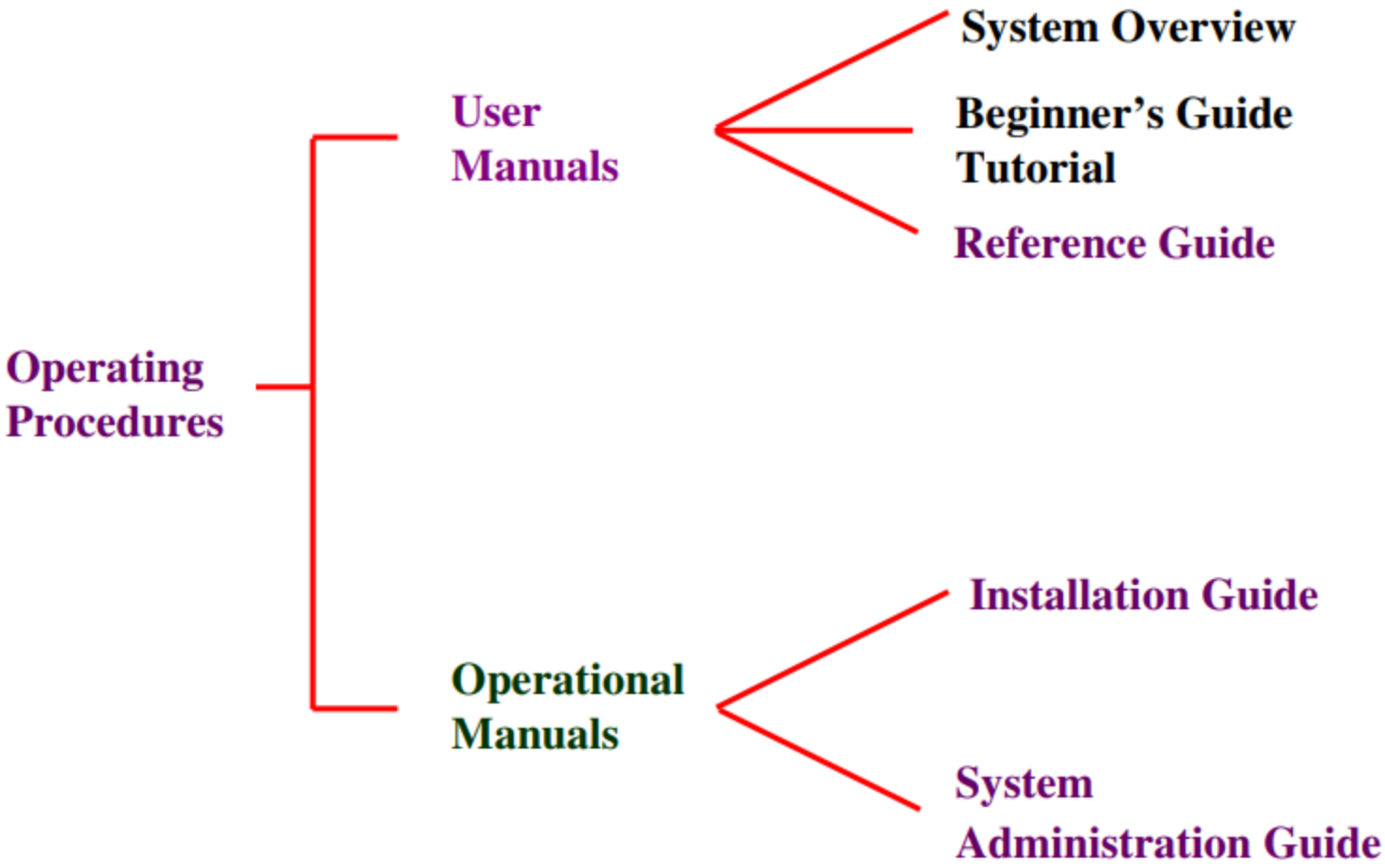
- *Software is more than programs.*
- *Software encompasses: (*Components of software)
  - *programs (computer programs) that when executed provide desired features, function, and performance;*
  - *Documentation consist of different types of manuals.*
  - *Operating Procedures consist of instructions to set up and use the software system and instructions on how to react to a system failure.*
  - Software=Program+Documentation+Operating Procedures

Components of software





List of documentation manuals



List of operating procedure manuals.

Software Engineering Definition

- "The establishment and use of sound engineering principles in order to obtain economically developed software that is reliable and works efficiently on real machines".
- "A discipline whose aim is the production of quality software, software that is delivered on time, within budget, and that satisfies its requirements.

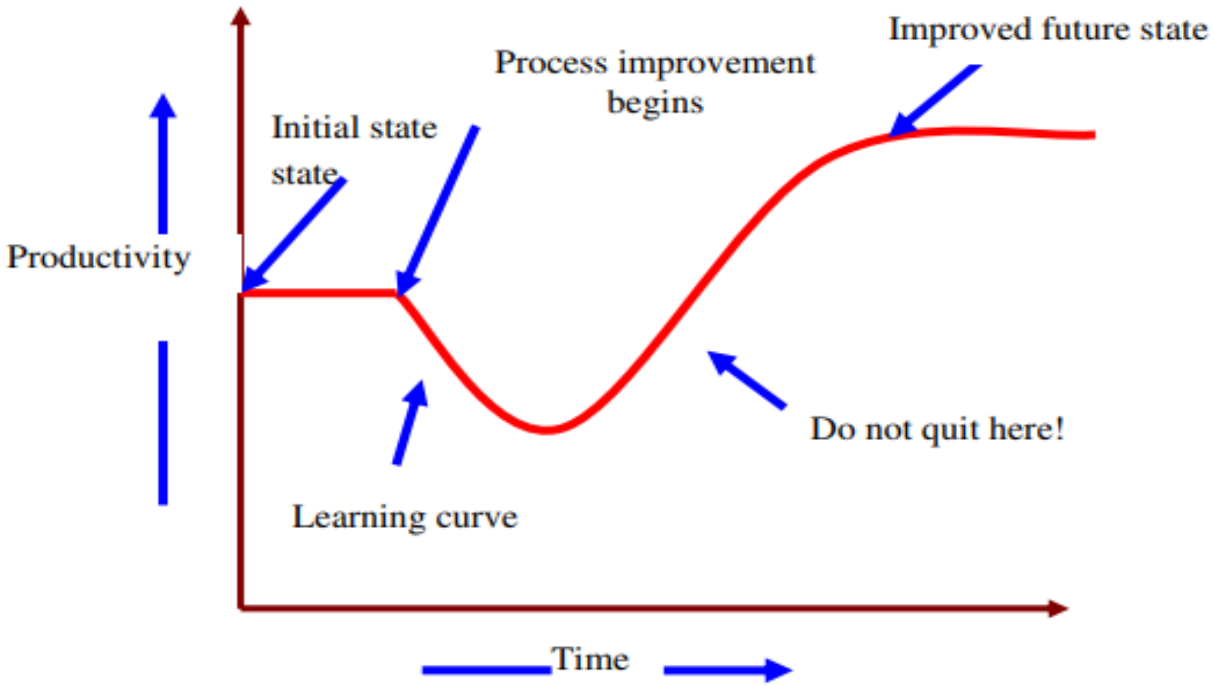
Why is software engineering needed?

- To predict time, effort, and cost
- To improve software quality
- To improve maintainability
- To meet increasing demands
- To lower software costs
- To successfully build large, complex software systems
- To facilitate group effort in developing software

Software Process

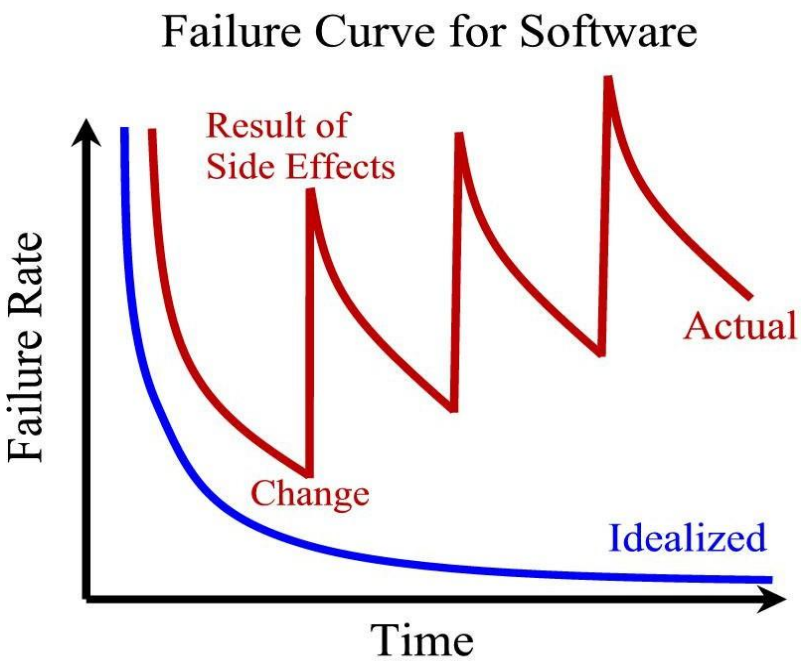
- The software process is the way in which we produce software.
- Why is it difficult to improve software process ?
  - Not enough time
  - Lack of knowledge
- Wrong motivations
- Insufficient commitment

Software Process

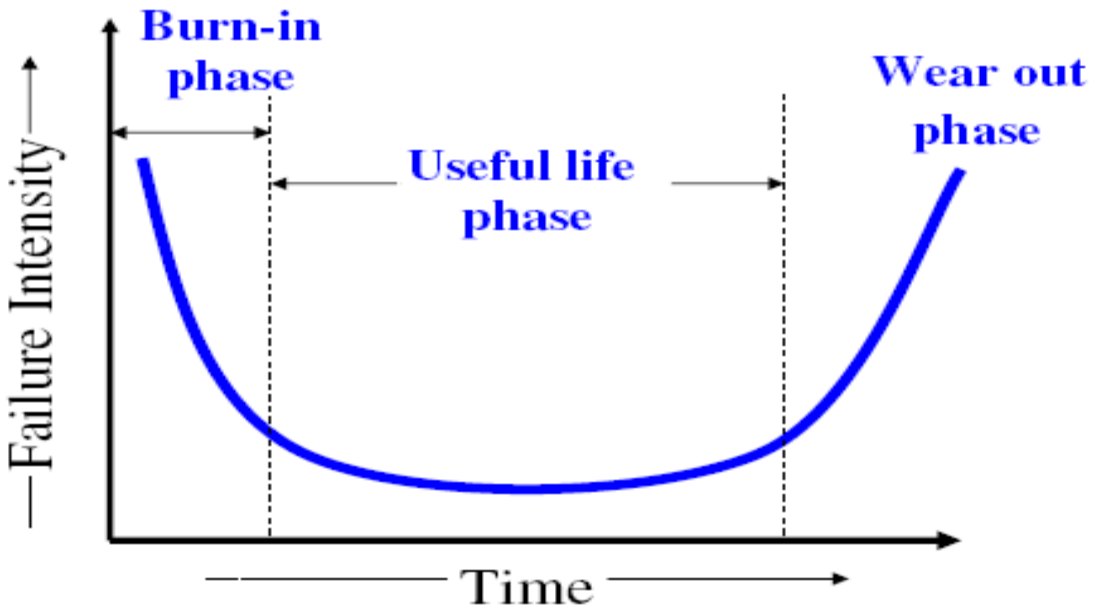


Software Characteristics

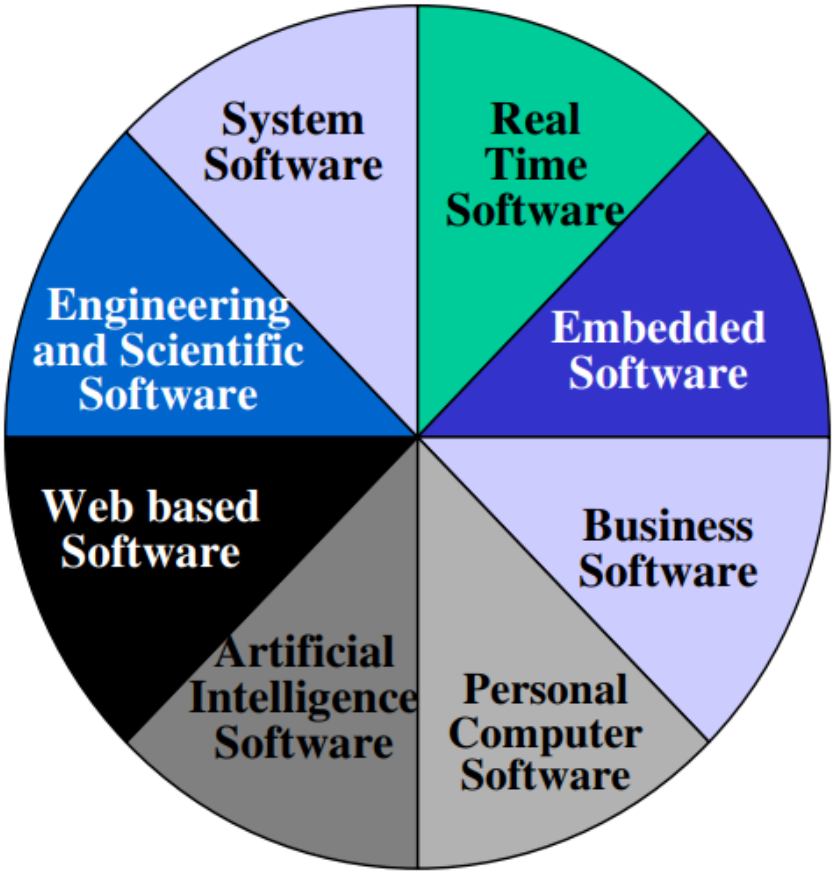
- Software is not manufactured.
- Reusability of components.
- Software is flexible.
- Software does not wear out.



Bath-Tub Curve for Hardware Failure



The changing Nature of Software



Software Myths

- Software is easy to change.
- Computers provide greater reliability than the devices they replace.
- Testing software or providing software correct can remove all the errors.
- Reusing software increases safety.
- Software can work right the first time.
- Software with more features is better software.
- The only deliverable for a software development project is the tested code.
- Aim is to develop working programs . *Now objective is to develop good quality maintainable programs!*

Some Terminologies-Deliverables and Milestones

- Different deliverables are generated during software development. The examples are source code, user manuals, operating procedure manuals etc.
- The milestones are the events that are used to ascertain the status of the project. Finalization of specification is a milestone. Completion of design documentation is another milestone. The milestones are essential for project planning and management.

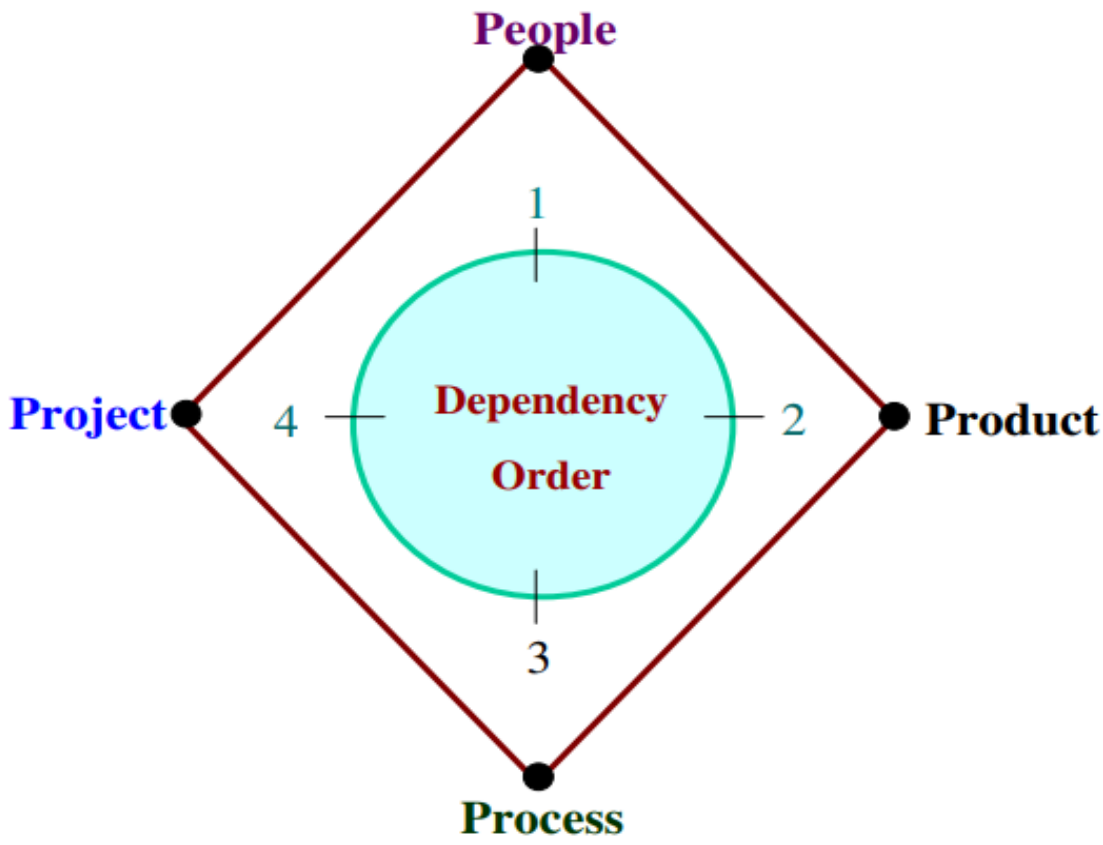
Product and Process

- **Product**: What is delivered to the customer, is called a product. It may include source code, specification document, manuals, documentation etc. Basically, it is nothing but a set of deliverables only.
- **Process**: Process is the way in which we produce software. It is the collection of activities that leads to (a part of) a product. An efficient process is required to produce good quality products.

Measures, Metrics and Measurement

- A **measure** provides a quantitative indication of the extent, dimension, size, capacity, efficiency, productivity or reliability of some attributes of a product or process.
- **Measurement** is the act of evaluating a measure.
- A **metric** is a quantitative measure of the degree to which a system, component or process possesses a given attribute.

Role of Management in Software Development-4 P's of Software Engg.



Refer Text Book for details.

MCQ

Multiple Choice Questions

Note: Select most appropriate answer of the following questions:

1.1 Software is
(a) Superset of programs (b) subset of programs
(c) Set of programs (d) none of the above

1.2 Which is NOT the part of operating procedure manuals?
(a) User manuals (b) Operational manuals
(c) Documentation manuals (d) Installation manuals

1.3 Which is NOT a software characteristic?
(a) Software does not wear out (b) Software is flexible
(c) Software is not manufactured (d) Software is always correct

1.4 Product is
(a) Deliverables (b) User expectations
(c) Organization's effort in development (d) none of the above

1.5 To produce a good quality product, process should be
(a) Complex (b) Efficient
(c) Rigorous (d) none of the above

Software Engineering (2nd ed.), By K.K. Aggarwal & Yogesh Singh, Copyright © New Age International Publishers, 2007

MCQ

Multiple Choice Questions

Note: Select most appropriate answer of the following questions:

1.6 Which is not a product metric?

(a) Size (b) Reliability
(c) Productivity (d) Functionality

1.7 Which is NOT a process metric?

(a) Productivity (b) Functionality
(c) Quality (d) Efficiency

1.8 Effort is measured in terms of:

(a) Person-months (b) Rupees
(c) Persons (d) Months

1.9 UML stands for

(a) Uniform modeling language (b) Unified modeling language
(c) Unit modeling language (d) Universal modeling language

1.1 An independently deliverable piece of functionality providing access to its services through interface is called

(a) Software measurement (b) Software composition
(c) Software measure (d) Software component

Software Engineering (3rd ed.), By K.K. Aggarwal & Yogesh Singh, Copyright © New Age International Publishers, 2007

MCQ

Multiple Choice Questions

Note: Select most appropriate answer of the following questions:

1.11 Infrastructure software are covered under
(a) Generic products (b) Customized products
(c) Generic and Customized products (d) none of the above

1.12 Management of software development is dependent on
(a) people (b) product
(c) process (d) all of the above

1.13 During software development, which factor is most crucial?
(a) People (b) Product
(c) Process (d) Project

1.14 Program is
(a) subset of software (b) super set of software
(c) software (d) none of the above

1.15 Milestones are used to
(a) know the cost of the project (b) know the status of the project
(c) know user expectations (d) none of the above

Software Engineering (3rd ed.), By K.K. Aggarwal & Yogesh Singh, Copyright © New Age International Publishers, 2007

MCQ

Multiple Choice Questions

Note: Select most appropriate answer of the following questions:

1.16 The term module used during design phase refers to

(a) Function (b) Procedure
(c) Sub program (d) All of the above

1.17 Software consists of

(a) Set of instructions + operating system
(b) Programs + documentation + operating procedures
(c) Programs + hardware manuals (d) Set of programs

1.18 Software engineering approach is used to achieve:

(a) Better performance of hardware (b) Error free software
(c) Reusable software (d) Quality software product

1.19 Concept of software engineering are applicable to

(a) Fortran language only (b) Pascal language only
(c) 'C' language only (d) All of the above

1.20 CASE Tool is

(a) Computer Aided Software Engineering (b) Component Aided Software Engineering
(c) Constructive Aided Software Engineering (d)Computer Analysis Software Engineering

Software Life Cycle Models

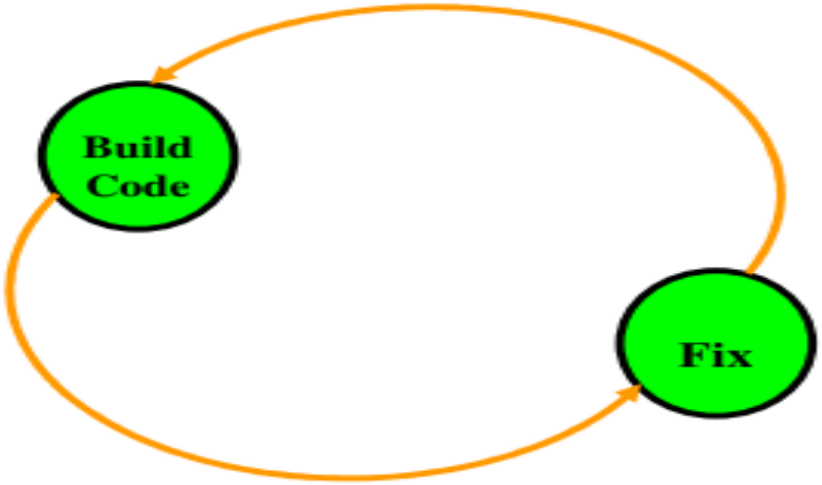
The goal of Software Engineering is to provide models and processes that lead to the production of well-documented maintainable software in a manner that is predictable.

Software Life Cycle Models

"The period of time that starts when a software product is conceived and ends when the product is no longer available for use. The software life cycle typically includes a requirement phase, design phase, implementation phase, test phase, installation and check out phase, operation and maintenance phase, and sometimes retirement phase".

Build & Fix Model

- Product is constructed without specifications or any attempt at design .
- Adhoc Approach and not well defined.
- Simple two phase model.



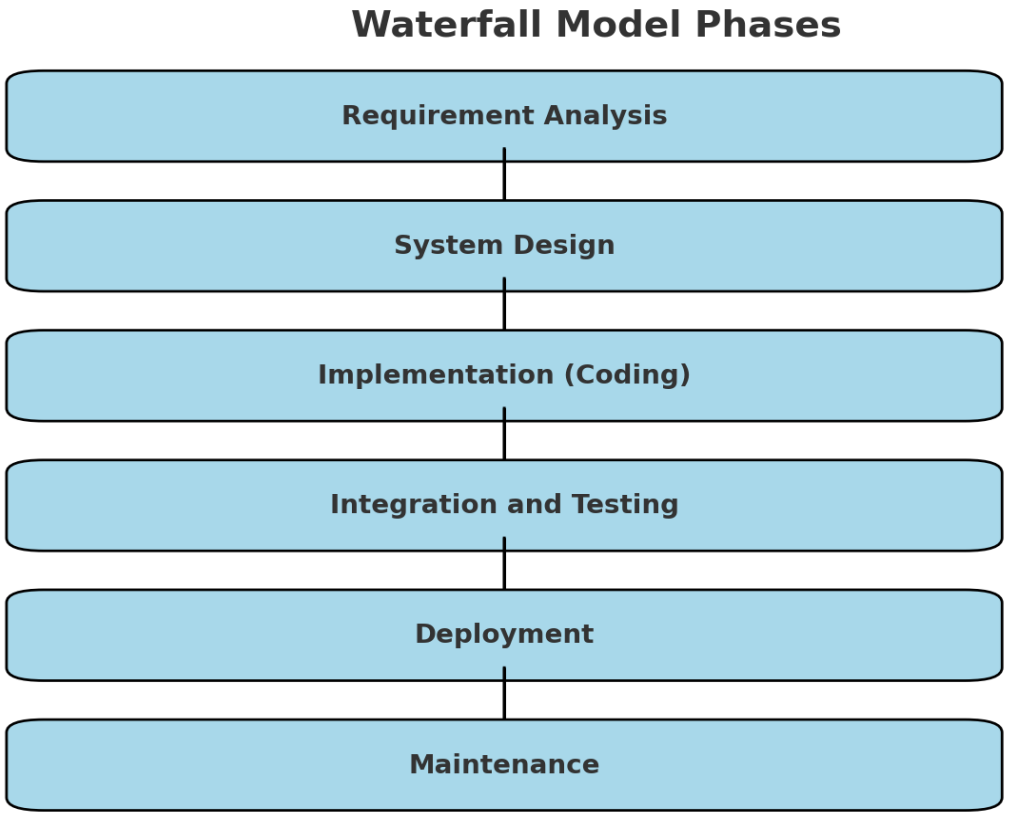
Build & Fix Model

- Two-Phase Model.
- The first phase is to write code.
- The next phase is to fix it. Fixing in this context may be error correction or addition of further functionality.

Build & Fix Model

- Suitable for small programming exercises of 100 or 200 lines.
- Unsatisfactory for software for any reasonable size.
- Code soon becomes unfixable & unenhanceable.
- No room for structured design.
- Cost is high.
- Maintenance is Extremely difficult.

Waterfall Model



Waterfall Model

- In "The Waterfall" approach, the whole process of software development is divided into separate phases. In Waterfall model, typically, the outcome of one phase acts as the input for the next phase sequentially.

- This model is easy to understand and reinforces the notion of "define before design" and "design before code". The model expects complete & accurate requirements early in the process, which is unrealistic.

1.Requirement analysis and specification Phase

The first phase of the Waterfall Model is requirement analysis, where the needs and expectations of stakeholders are gathered and documented. This involves extensive discussions with clients, end-users, and other stakeholders to ensure a clear understanding of what the software must accomplish. The result of this phase is a comprehensive Software Requirement Specification (SRS) document that serves as a reference for all subsequent phases.

2. System Design

- In the system design phase, the focus is on translating the requirements from the SRS into a blueprint for the software's architecture. This phase includes creating high-level designs (HLD) to define the system's overall structure and low-level designs (LLD) for module-level details. Design outputs often include data flow diagrams, Unified Modeling Language (UML) diagrams, and database schemas, providing a detailed guide for the development team.

3. Implementation (Coding)

- The implementation phase involves writing and compiling code to create the actual software based on the design documents. Developers work on coding individual modules and ensuring they function as expected through unit testing. Once the modules are developed, they are integrated into a complete system. This phase transforms the theoretical design into functional components of the software.

4. Integration and Testing

- In this phase, the developed modules are integrated, and the software is tested to verify its functionality, performance, and reliability. Different levels of testing, such as unit testing, integration testing, and system testing, are conducted to identify and fix defects. The primary goal of this phase is to ensure that the software operates as intended and meets the requirements specified in the SRS.

5. Deployment

- The deployment phase involves installing the software in the production environment and making it available to end-users. Before deployment, user acceptance testing (UAT) is performed to confirm that the software meets user needs. Once approved, the software is deployed, and any required configurations are completed to ensure a smooth transition to the operational stage.

6. Maintenance

- The final phase of the Waterfall Model is maintenance, which ensures the software continues to perform effectively after deployment. This phase includes activities such as fixing bugs, making updates to accommodate new requirements, and adapting to changes in the operating environment. Maintenance ensures the software remains relevant, reliable, and up-to-date throughout its lifecycle.

Waterfall Model

5. **Deployment of system:** Once the functional and non functional testing is done, the product is deployed in the customer environment or released into the market.
6. **Operation and Maintenance:** There are some issues which come up in the client environment. To fix those issues patches are released. Also to enhance the product some better versions are released. Maintenance is done to deliver these changes in the customer environment.

Waterfall Model

Every software developed is different and requires a suitable SDLC approach to be followed based on the internal and external factors. Some situations where the use of Waterfall model is most appropriate are:

- Requirements are very well documented, clear and fixed.
- Product definition is stable.
- Technology is understood and is not dynamic.
- There are no ambiguous requirements.
- Ample resources with required expertise are available to support the product.
- The project is short.

Problems of Waterfall Model

- It is difficult to define all requirements at the beginning of a project.
- This model is not suitable for accommodating any change.
- A working version of the system is not seen until late in the project's life.
- It does not scale up well to large projects.
- Real projects are rarely sequential.

Incremental Models

- They are effective in the situations where requirements are defined precisely and there is no confusion about the functionality of the final product.
- After every cycle a useable product is given to the customer.
- Popular particularly when we have to quickly deliver a limited functionality system.
- For eg , in the university automation software Library automation ,module will be delivered in the first phase and examination automation module in the second phase and as so on.

Incremental Process Model

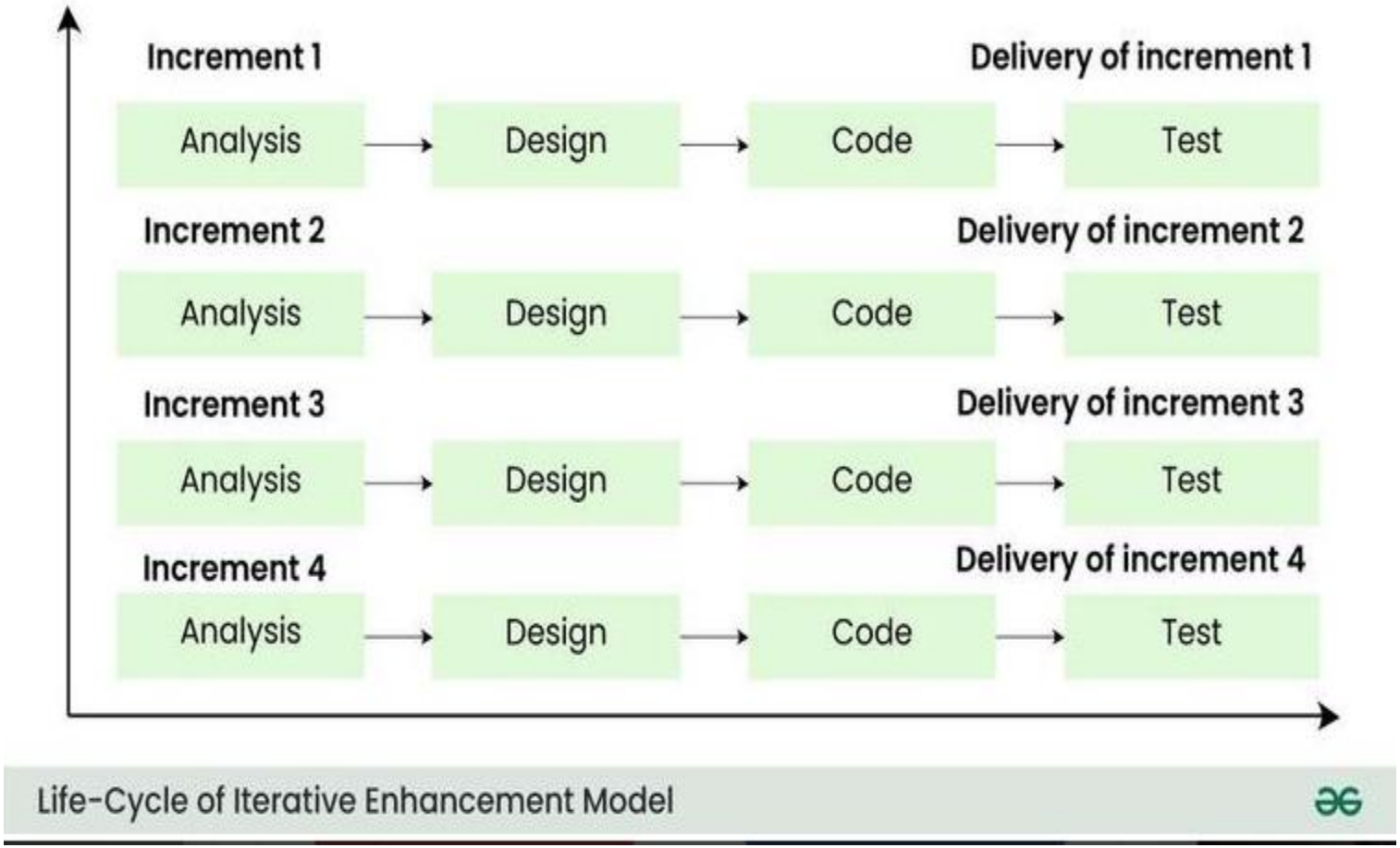
Software is not delivered all at once. Instead, development is divided into multiple increments, and each increment goes through requirements, design, development, testing, and implementation phases. Every new release improves the previous version.

- **Step-by-Step Development:** The system is built gradually in small parts.
- **Early Working Software:** A usable version is delivered early.
- **Customer Feedback:** Feedback after each increment improves future releases.
- **Flexible to Changes:** New features can be added in later increments.
- **Structured yet Iterative:** Combines planning of Waterfall with flexibility of iteration.

Iterative Enhancement Model

- This model has the same phases as the waterfall model, but with fewer restrictions. Generally the phases occur in the same order as in the waterfall model, but they may be conducted in several cycles. Useable product is released at the end of the each cycle, with each release providing additional functionality.
- Customers and developers specify as many requirements as possible and prepare a SRS document.
- Developers and customers then prioritize these requirements.
- Developers implement the specified requirements in one or more cycles of design, implementation and test based on the defined priorities.

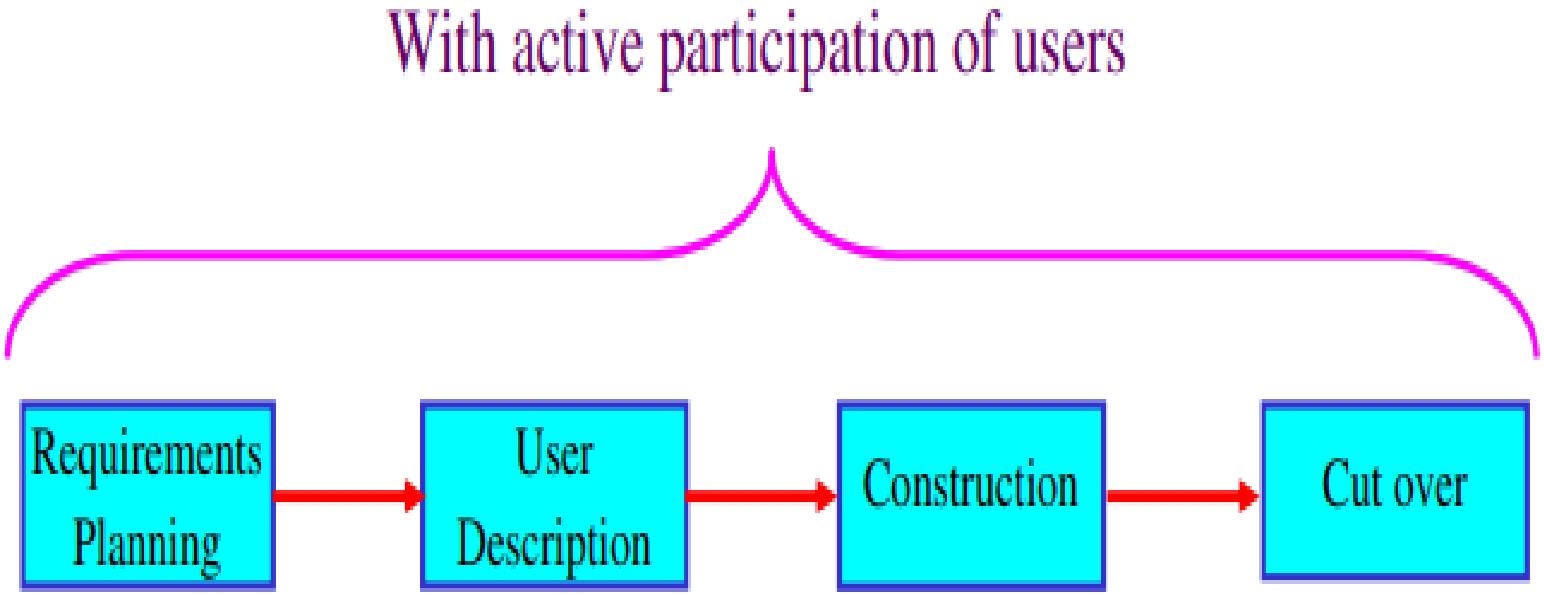
Iterative Enhancement Model



RAID-Rapid application Development Model

- This model is an incremental model.
- Developed by IBM in 1980s.
- Here User participation is essential from requirement phase to delivery of the product.
- The process is stated with building a rapid prototype and it is given to user for evaluation.
- The user feedback is obtained and prototype is refined.
- The process continues, till the requirements' are finalized.
- SRS and design documents are prepared with the association of users.

RAID


With active participation of users
Requirements Planning
User Description
Construction
Cut over

The 4 Phases of the RAD Model

1. **Requirements Planning**

   Involves gathering **high-level requirements** and defining the project scope. Stakeholders (users, developers, and clients) discuss objectives and constraints. Unlike traditional models, this phase is **brief** and **flexible**.

2. **User Description**

   A continuous **interactive process** between developers and users. Prototypes (mockups, wireframes, or working models) are built and **refined** based on feedback. Users actively participate to ensure the system meets their needs.

The 4 Phases of the RAD Model

3. Construction Phase

Developers build the **actual system** using reusable components and rapid development tools. Frequent iterations and user feedback ensure quick modifications. This phase includes **coding, integration, and testing**.

4. Cut Over Phase

The **final product** is tested, deployed, and released to end users. Training, support, and maintenance plans are put in place. Unlike traditional models, RAD focuses on **shorter implementation cycles**.

Key Advantages of the RAD Model

- Faster development with continuous user involvement.
- Encourages adaptability and quick iterations.
- Reduces risk by integrating early user feedback.

Limitations:-

- Requires Active User Involvement
- Not Suitable for Large-Scale or Complex Systems
- High Dependence on Skilled Developers
- Difficult to Manage Changing Scope
- Higher Development Costs

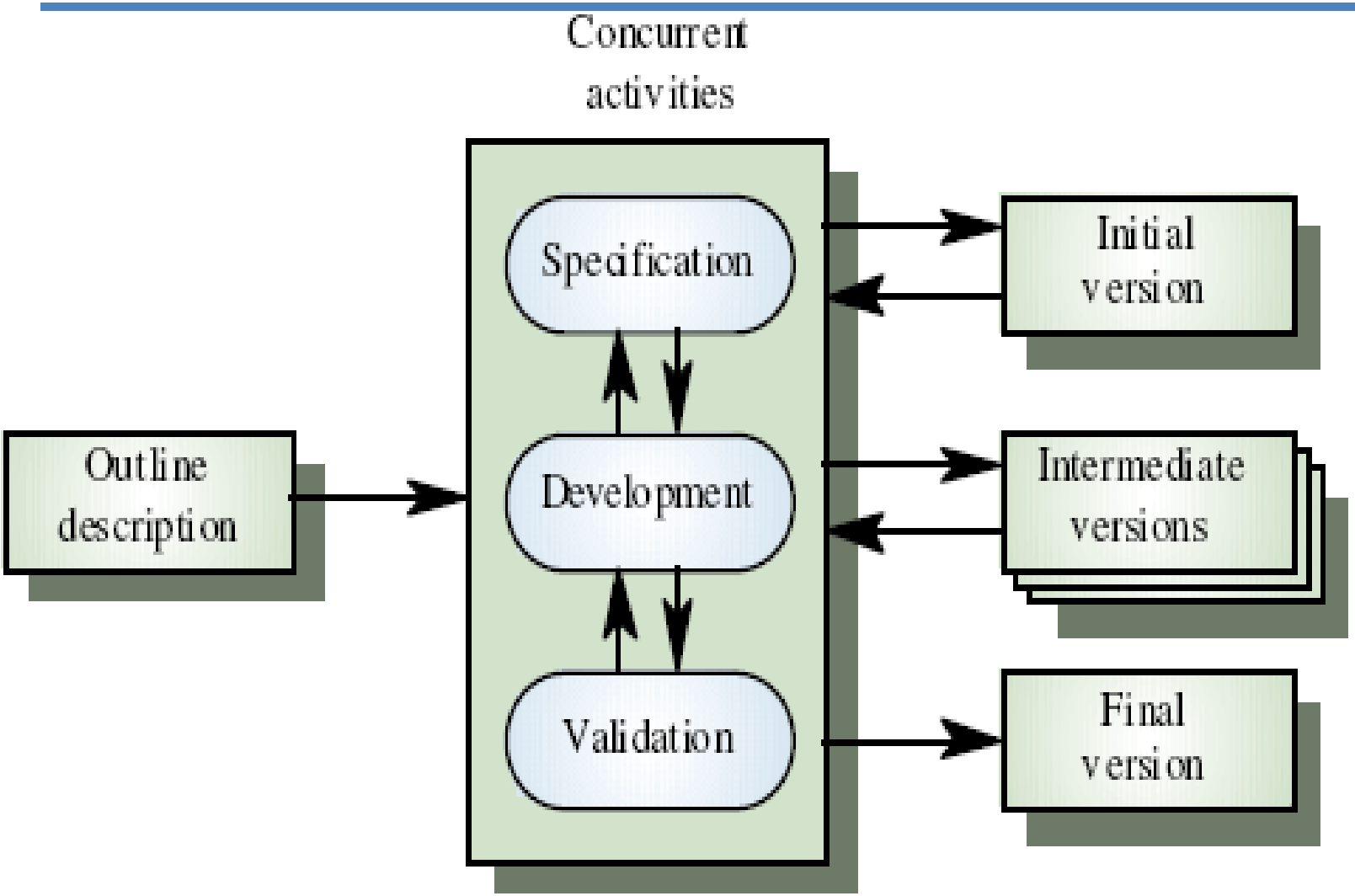
Evolutionary Process Models

- Evolutionary process model resembles iterative enhancement model. The same phases as defined for the waterfall model occur here in a cyclical fashion.
- This model differs from iterative enhancement model in the sense that this does not require a useable product at the end of each cycle.
- In evolutionary development, requirements are implemented by category rather than by priority.
- This model is useful for projects using new technology that is not well understood.
- This is also used for complex projects where all functionality must be delivered at one time, but the requirements are unstable or not well understood at the beginning.

Evolutionary Process Models

- For example, in a simple database application, one cycle implement the graphical user Interface (GUI), another file manipulation, another queries and another updates.
- All the four phases must complete before there is working product available.
- In contrast , an iterative enhancement model would start by developing a very simplistic, but usable database. On the completion of each cycle, the system would become very sophisticated. However it provide all the critical functionality by the end of first cycle.

Evolutionary Process Models



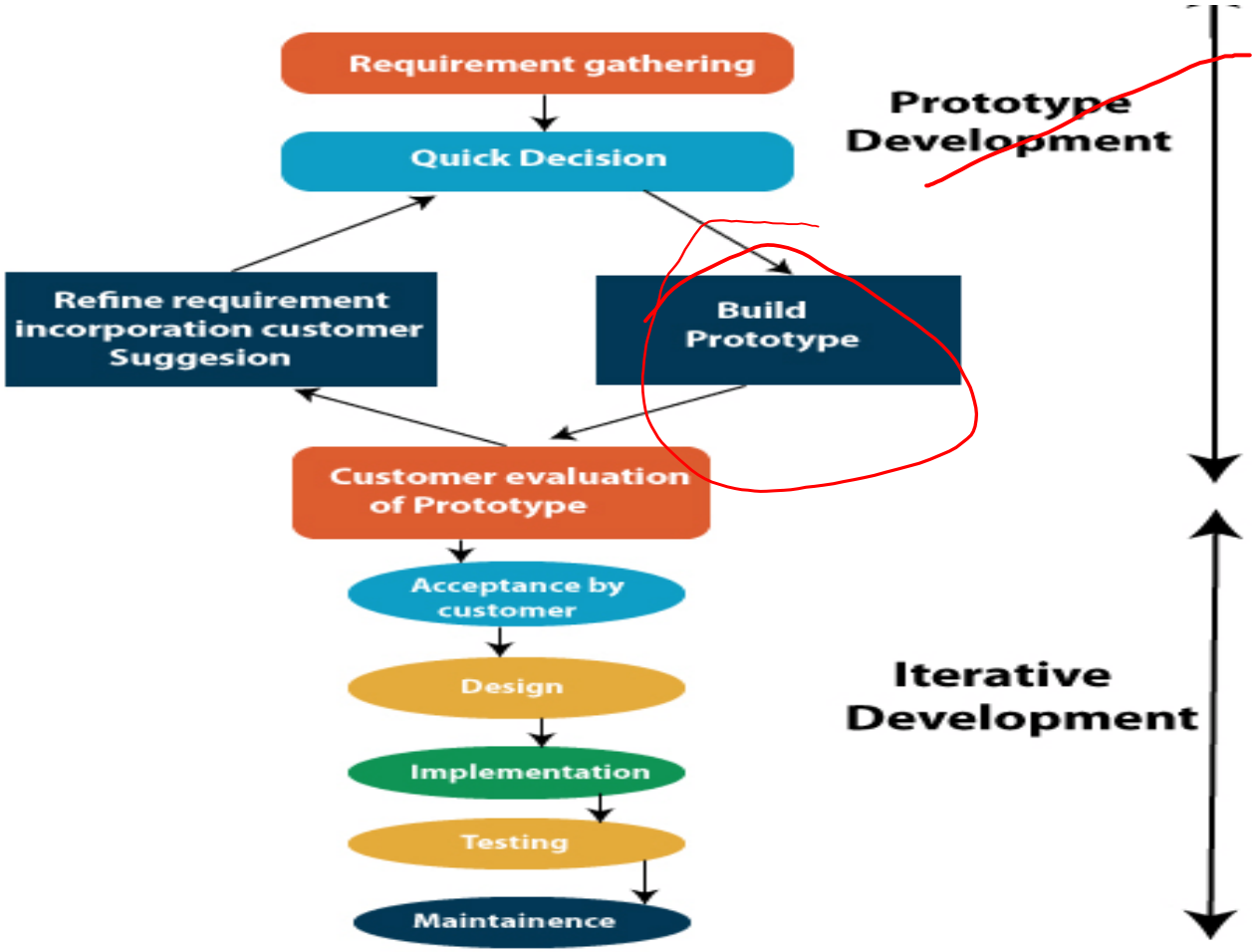
Evolutionary Process Models

- **1. Prototyping Model**
- **2. Spiral Model**

Prototyping Model

- A disadvantage of water model model is that the working software is not available until late in the process, thus delaying the discovery of serious errors.
- An alternative is to develop a working prototype of the software instead of developing the actual software.
- The working prototype is developed as per current available requirements.
- Basically it has limited functional capabilities, low reliability and untested performance (usually low).

Prototyping Model



Prototyping Model

- The prototype may be a usable program but is not suitable as the final software product.
- The code for the prototype is thrown away. However experience gathered helps in developing the actual system.
- The development of a prototype might involve extra cost, but overall cost might turnout to be lower than that of an equivalent system developed using the waterfall model.

Prototyping Model

- The developers should develop prototype as early as possible to speed up software development process.
- After all, the sole use is to determine customer's real needs.
- After the finalization of SRS document, the prototype is discarded and actual system is then developed using waterfall approach.
- This model requires extensive participation and involvement of the customer, which is not always possible.

Advantage of Prototype Model

- Reduce the risk of incorrect user requirement
- Good where requirement are changing/uncommitted
- Regular visible process aids management
- Support early product marketing
- Reduce Maintenance cost.
- Errors can be detected much earlier as the system is made side by side.

Disadvantage of Prototype Model

- An unstable/badly implemented prototype often becomes the final product.
- Require extensive customer collaboration
  - Costs customer money
  - Needs committed customer
  - Difficult to finish if customer withdraw
  - May be too customer specific, no broad market
- Difficult to know how long the project will last.
- Easy to fall back into the code and fix without proper requirement analysis, design, customer evaluation, and feedback.
- Prototyping tools are expensive.
- Special tools & techniques are required to build a prototype.
- It is a time-consuming process.

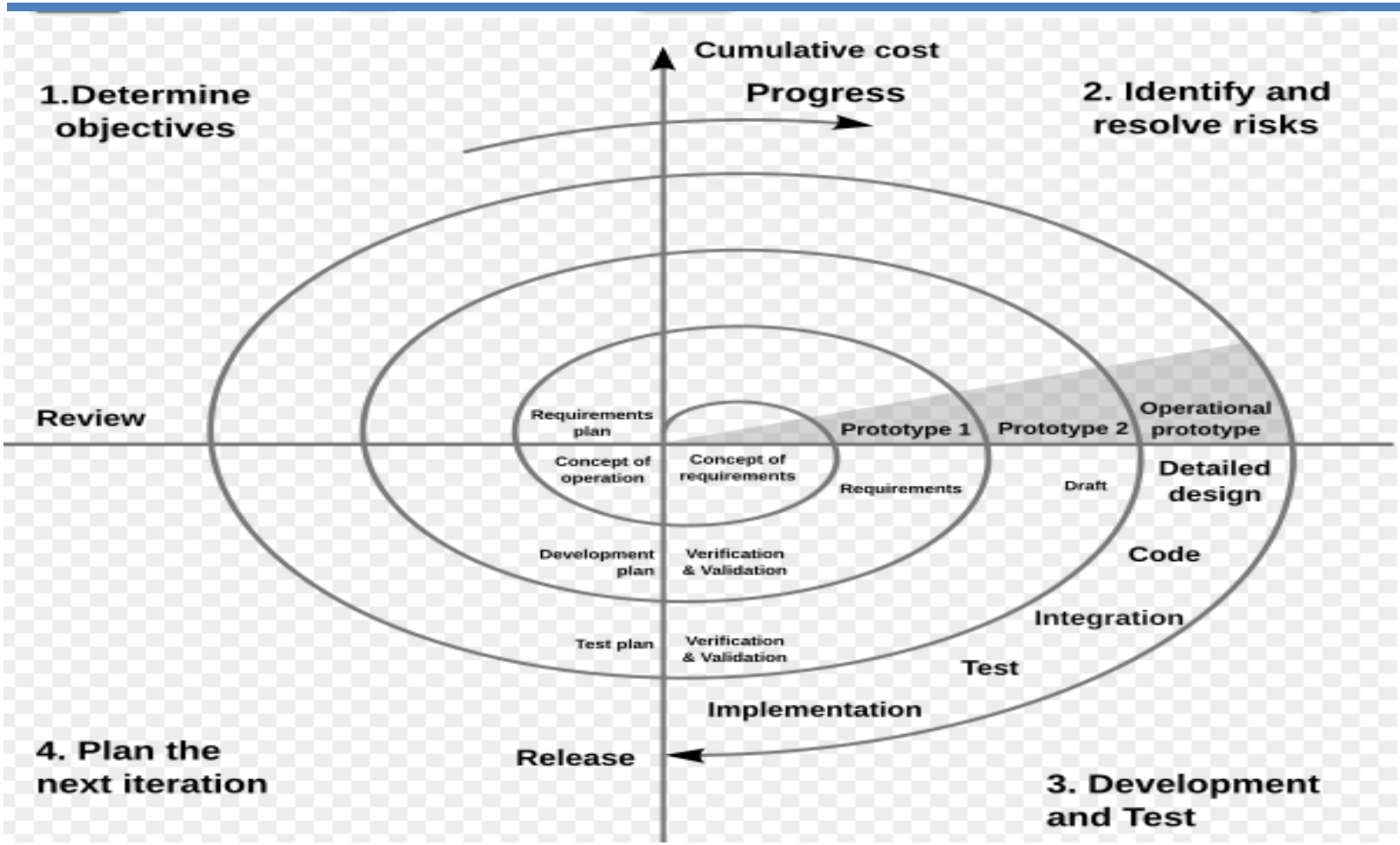
Spiral Model

- Models do not deal with uncertainly which is inherent to software projects.
- Important software projects have failed because project risks were neglected & nobody was prepared when something unforeseen happened.
- Barry Boehm recognized this and tired to incorporate the "project risk" factor into a life cycle model.
- The result is the spiral model, which was presented in 1986.
- The **Spiral Model** is a **risk-driven** software development process model that combines iterative development with elements of the **Waterfall** model. It was proposed by **Barry Boehm** in 1986 and is particularly useful for large, complex, and high-risk projects.

Spiral Model

- The radial dimension of the model represents the cumulative costs. Each path around the spiral is indicative of increased costs. The angular dimension represents the progress made in completing each cycle. Each loop of the spiral from X-axis clockwise through 360 degree represents one phase. One phase is split roughly into four sectors of major activities.
- **Planning:** Determination of objectives, alternatives & constraints.
- **Risk Analysis:**. Analyze alternatives and attempts to identify and resolve the risks involved
- **Development:** Product development and testing product.
- **Assessment:** Customer evaluation

Spiral Model



Spiral Model

- An important feature of the spiral model is that each phase is completed with a review by the people concerned with the project (designers and programmers)
- The advantage of this model is the wide range of options to accommodate the good features of other life cycle models.
- It becomes equivalent to another life cycle model in appropriate situations.
- The spiral model has some difficulties that need to be resolved before it can be a universally applied life cycle model. These difficulties include lack of explicit process guidance in determining objectives, constraints, alternatives; relying on risk assessment expertise; and provides more flexibility than required for many applications.

When to Use the Spiral Model?

- When project **risks** need careful management.
- For **complex and large** software projects.
- When frequent **customer feedback** is required.
- In projects with evolving **requirements.**

Advantages of the Spiral Model

✓ **Risk Management:** Helps in early detection and mitigation of risks.

✓ **Iterative Development:** Continuous refinement of the software.

✓ **Flexible Requirements:** Allows changes in requirements at different stages.

✓ **Customer Involvement:** Frequent feedback leads to better software.

Disadvantages of the Spiral Model

- ❌ **Expensive:** Costly due to risk analysis and iterative cycles.
- ❌ **Complex Process:** Requires experienced personnel.
- ❌ **Time-Consuming:** Multiple iterations make it longer compared to simpler models.

Agile Model

- In earlier days, the **Iterative Waterfall Model** was very popular for completing a project. But nowadays, developers face various problems while using it to develop software. The main difficulties included handling customer change requests during project development and the high cost and time required to incorporate these changes. To overcome these drawbacks of the Waterfall Model, in the mid-1990s the **Agile Software Development** model was proposed.

Agile Model

- **The Agile Model** was primarily designed to help a project adapt quickly to change requests. So, the main aim of the Agile model is to facilitate quick project completion. To accomplish this task, agility is required. Agility is achieved by fitting the process to the project and removing activities that may not be essential for a specific project. Also, anything that is a waste of time and effort is avoided. The Agile Model refers to a group of development processes.
- Agile development requires a high degree of collaboration and communication among team members, as well as a willingness to adapt to changing requirements and feedback from customers.

Agile

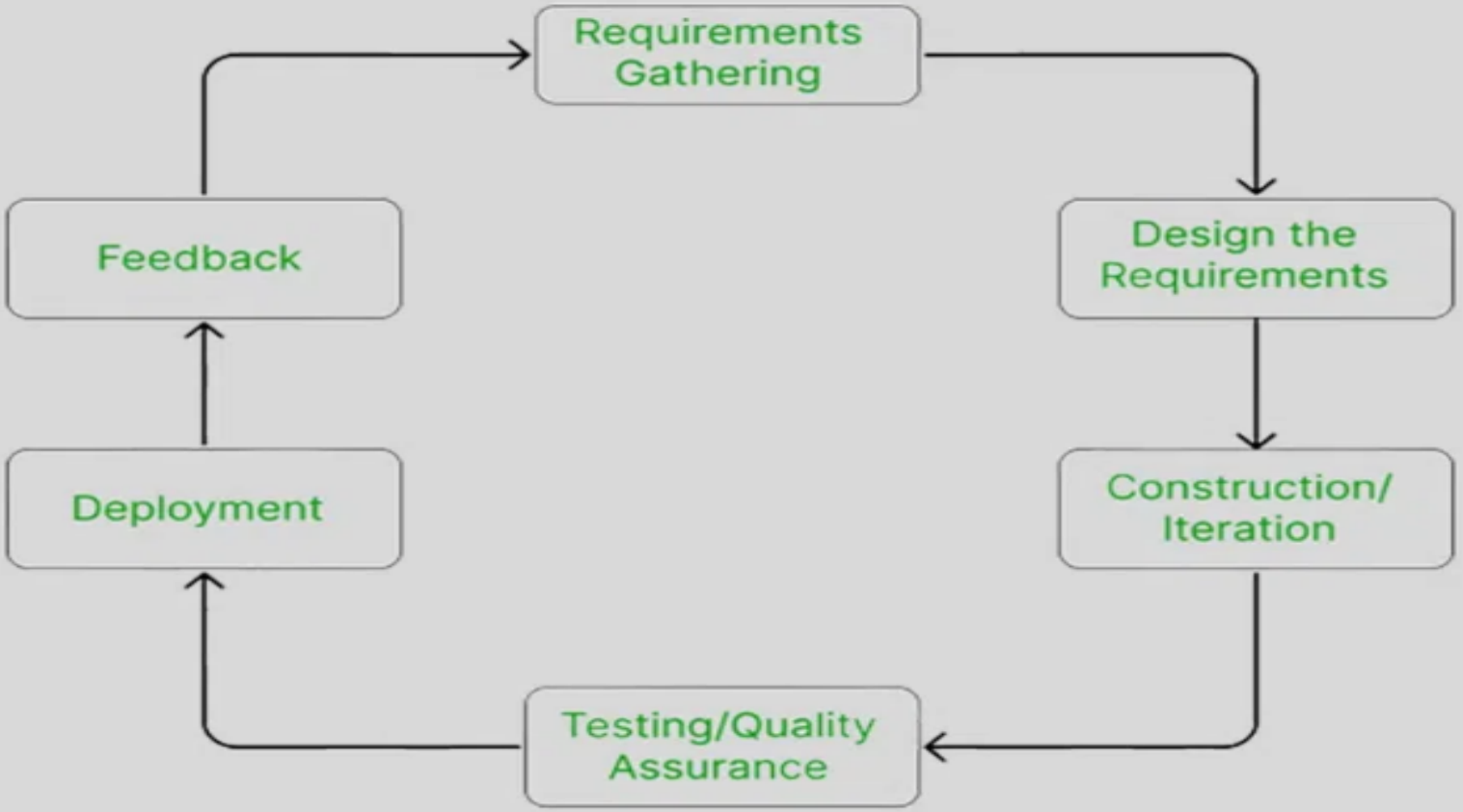


Fig:- Steps in Agile SDLC Model

Phases of Agile Model

- The agile model is a combination of iterative and incremental process models. The steps involve in agile SDLC models are:
- Requirement gathering
- Design the Requirements
- Construction / Iteration
- Testing / Quality Assurance
- Deployment
- Feedback

Agile Model

- In the Agile model, the requirements are decomposed into many small parts that can be incrementally developed. The Agile model adopts Iterative development. Each incremental part is developed over an iteration. Each iteration is intended to be small and easily manageable and can be completed within a couple of weeks only. At a time one iteration is planned, developed, and deployed to the customers. Long-term plans are not made.

Advantages of Agile

- **Faster delivery** with working software at the end of each iteration.
- **Better collaboration** between teams and stakeholders.
- **Flexibility** to adapt to changing requirements.
- **Improved quality** through continuous testing and feedback.

Disadvantages of Agile

- Requires **high customer involvement**.
- Can be **difficult to predict costs and timelines** upfront.
- May lead to **scope creep** due to evolving requirements.
- Needs **experienced teams** to manage self-organization effectively.

Selection of a Software Development Life Cycle (SDLC) Model

- Choosing the right **Software Development Life Cycle (SDLC) model** is crucial for the success of a project. The selection depends on several factors, including project requirements, complexity, timeline, and team expertise. Below are key considerations and a comparison of popular models to help in decision-making.

Key Factors for Selecting an SDLC Model

1. Project Size & Complexity

- Small, simple projects → **Waterfall, Prototype**
- Large, complex projects → **Agile, Spiral, V-Model**

2. Requirement Stability

- Well-defined, stable requirements → **Waterfall, V-Model**
- Evolving or unclear requirements → **Agile, Spiral, RAD**

3. Customer Involvement

- High customer involvement → **Agile, Prototype, RAD**
- Low customer involvement → **Waterfall, V-Model**

Key Factors for Selecting an SDLC Model

4. Time Constraints & Speed

- Fast development required → **RAD, Agile**
- Can afford longer planning and development → **Waterfall, Spiral**

5. Budget Constraints

- Fixed budget → **Waterfall, V-Model**
- Flexible budget for evolving needs → **Agile, Spiral**

6. Risk & Uncertainty

- High-risk projects → **Spiral, Agile**
- Low-risk projects → **Waterfall, V-Model**

Introduction

- After the finalization of SRS, we would like to estimate size, cost and development time of the project. Also, in many cases, customer may like to know the cost and development time even prior to finalization of the SRS.

- In order to conduct a successful software project, we must understand:

- Scope of work to be done
- The risk to be incurred
- The resources required
- The task to be accomplished
- The cost to be expended
- The schedule to be followed

Software Project Planning

- Software planning begins before technical work starts, continues as the software evolves from concept to reality, and culminates only when the software is retired.

Activities during Software Project Planning

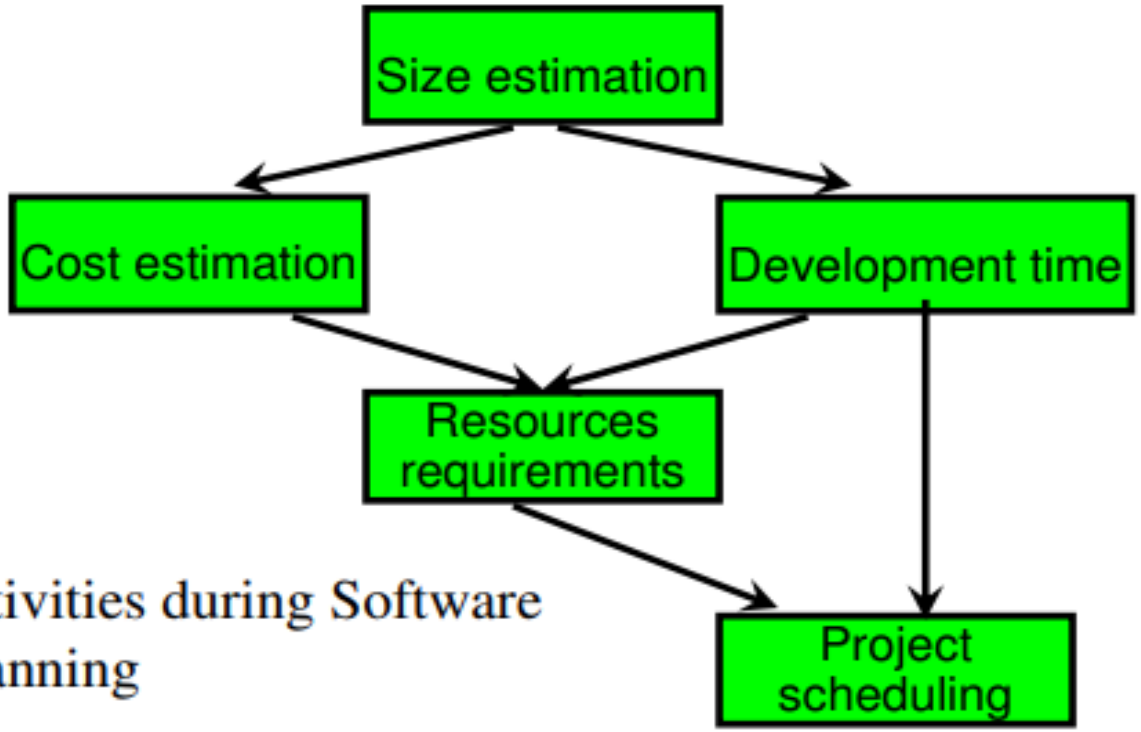


Fig. 1: Activities during Software Project Planning

Size Estimation

- Size estimation refers to the process of predicting or approximating the dimensions, magnitude, or extent of an object, task, or system. In various fields such as construction, manufacturing, software development, and logistics, accurate size estimation is essential for planning, resource allocation, budgeting, and scheduling. It involves evaluating factors like length, area, volume, effort, or complexity, depending on the context. Effective size estimation helps organizations optimize processes, prevent cost overruns, and ensure projects are completed within expected timelines. Although size estimation can be based on historical data, expert judgment, or mathematical models, it often requires adjustments as new information becomes available, making it a continuous and iterative process.

Lines of Code (LOC)

- This was the first measurement attempted.
- It has the advantage of being easily recognisable,seen and therefore counted.
- A line of code is any line of program text that is not a comment or blank line, regardless of the number of statements or fragments of statements on the line. This specifically includes all lines containing program header, declaration, and executable and non-executable statements".
- If LOC is simply a count of the number of lines then figure shown below contains 18 LOC
- When comments and blank lines are ignored, the program in figure 2 shown below contains 17 LOC

Example

Fig. 2: Function for sorting an array

```
int. sort (int x[ ], int n)
{
int i, j, save, im1;
/*This function sorts array x in ascending order */
If (n<2) return 1;
for (i=2; i<=n; i++)
{
im1=i-1;
for (j=1; j<=im; j++)
if (x[i] < x[j])
{
Save = x[i];
x[i] = x[j];
x[j] = save;
}
}
return 0;
}
```

Disadvantages of LOC

- LOC is Language dependent .
- A line of assembler is not same as a line of COBOL.
- It reflects only what system is rather than what it does.
- Measuring system by LOC is rather like measuring a building by the number of bricks involved in the construction.
- LOC focuses only on quantity, not the quality, efficiency, or maintainability of the code.
- **Ignores Complexity:** A small piece of code can be far more complex than a long one, but LOC won't capture this difference.

Function Count

- Alan Albrecht while working for IBM, recognized the problem in size measurement in the 1970s, and developed a technique (which he called Function Point Analysis), which appeared to be a solution to the size measurement problem.
- It measures functionality from the users point of view, that is on the basis of what the user requests and receives in return.
- In other words, two identical counting systems, one written in 4GL and the other in assembler, would have the same function count.

Function Count

- This makes sense to the user, because the object is to buy an accounting system ,not lines of assembler and it makes sense to the IT department ,because they can measure the performance differences between the assembler and 4GL environment.
- Uses the functionality of the software as a measure of its complexity.
- Analogy: For a given house, we can say how many square meters it has (LOC) or we can say how many bedrooms and bathrooms it has (FP).

Function Point analysis

The principle of Albrecht's function point analysis (FPA) is that a system is decomposed into functional units.

- **Inputs** :information entering the system.
- **Outputs** :information leaving the system.
- **Enquiries:** requests for instant access to information.
- **Internal logical files:** information held within the system.
- **External interface files**: information held by other systems that is used by the system being analyzed.

Function Points Units

The FPA functional units are shown in figure given below:

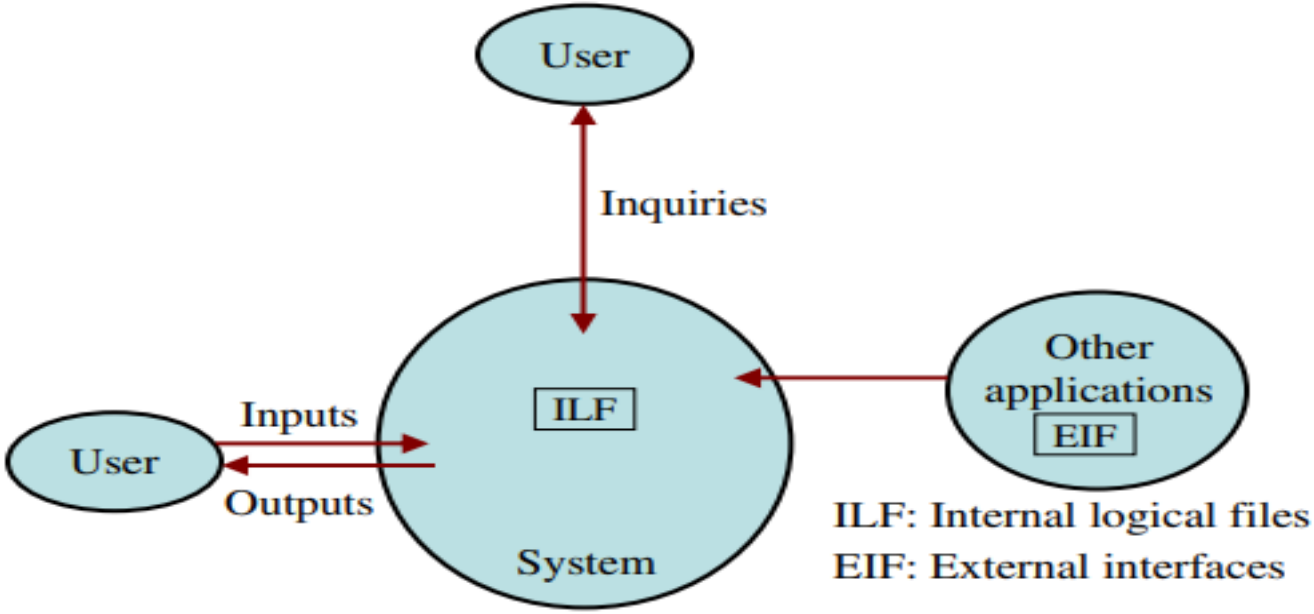


Fig. 3: FPAs functional units System

Functional Units

The five functional units are divided in two categories:

(i) Data function types

- Internal Logical Files (ILF): A user identifiable group of logical related data or control information maintained within the system.
- External Interface files (EIF): A user identifiable group of logically related data or control information referenced by the system, but maintained within another system. This means that EIF counted for one system, may be an ILF in another system.

Functional Units

(ii) **Transactional function types**

- External Input (EI): An EI processes data or control information that comes from outside the system. The EI is an elementary process, which is the smallest unit of activity that is meaningful to the end user in the business.
- External Output (EO): An EO is an elementary process that generate data or control information to be sent outside the system.
- External Inquiry (EQ): An EQ is an elementary process that is made up to an input-output combination that results in data retrieval.

Special Features

- Function point approach is independent of the language, tools, or methodologies used for implementation; i.e. they do not take into consideration programming languages, data base management systems, processing hardware or any other data base technology.
- Function points can be estimated from requirement specification or design specification, thus making it possible to estimate development efforts in early phases of development.

Special Features

- Function points are directly linked to the statement of requirements; any change of requirements can easily be followed by a re-estimate.
- Function points are based on the system user's external view of the system, non-technical users of the software system have a better understanding of what function points are measuring.

Counting Function Points

Counting function points

Functional Units	Weighting factors		
	Low	Average	High
External Inputs (EI)	3	4	6
External Output (EO)	4	5	7
External Inquiries (EQ)	3	4	6
External logical files (ILF)	7	10	15
External Interface files (EIF)	5	7	10

Table 1 : Functional units with weighting factors

Unadjusted Function Point

Table 2: UFP calculation table

Functional Units	Count Complexity			Complexity Totals	Functional Unit Totals
External Inputs (EIs)		Low x 3	=		
		Average x 4	=		
		High x 6	=		
External Outputs (EOs)		Low x 4	=		
		Average x 5	=		
		High x 7	=		
External Inquiries (EQs)		Low x 3	=		
		Average x 4	=		
		High x 6	=		
External logical Files (ILFs)		Low x 7	=		
		Average x 10	=		
		High x 15	=		
External Interface Files (EIFs)		Low x 5	=		
		Average x 7	=		
		High x 10	=		
Total Unadjusted Function Point Count					

Unadjusted Function Point

- The weighting factors are identified for all functional units and multiplied with the functional units accordingly. The procedure for the calculation of Unadjusted Function Point (UFP) is given in table shown on next slide.
- The procedure for the calculation of UFP in mathematical form is given in next slide.

Unadjusted Function Point

$$UFP = \sum_{i=1}^{5} \sum_{J=1}^{3} Z_{ij} w_{ij}$$

Where i indicate the row and j indicates the column of Table 1

W_{ij} : It is the entry of the ith row and j^{th} column of the table 1

Zij : It is the count of the number of functional units of Type *i* that have been classified as having the complexity corresponding to column *j*.

- Organizations that use function point methods develop a criterion for determining whether a particular entry is Low, Average or High. Nonetheless, the determination of complexity is somewhat subjective.

$$\mathbf{FP = UFP * CAF}$$

- Where CAF is complexity adjustment factor and is equal to [0.65 +0.01 x ΣFi]. The Fi (*i*=1 to 14) are the degree of influence and are based on responses to questions noted in table .

Computing Function Points

Table 3 : Computing function points.

Rate each factor on a scale of 0 to 5.

0	1	2	3	4	5
No Influence	*Incidental*	*Moderate*	*Average*	*Significant*	*Essential*

Number of factors considered (F_i)

1. Does the system require reliable backup and recovery ?
2. Is data communication required ?
3. Are there distributed processing functions ?
4. Is performance critical ?
5. Will the system run in an existing heavily utilized operational environment ?
6. Does the system require on line data entry ?
7. Does the on line data entry require the input transaction to be built over multiple screens or operations ?
8. Are the master files updated on line ?
9. Is the inputs, outputs, files, or inquiries complex ?
10. Is the internal processing complex ?
11. Is the code designed to be reusable ?
12. Are conversion and installation included in the design ?
13. Is the system designed for multiple installations in different organizations ?
14. Is the application designed to facilitate change and ease of use by the user ?

Uses of Function Points

Functions points may compute the following important metrics:

Productivity	=	FP / persons-months
Quality	=	Defects / FP
Cost	=	Rupees / FP
Documentation	=	Pages of documentation per FP

These metrics are controversial and are not universally acceptable. There are standards issued by the International Functions Point User Group (IFPUG, covering the Albrecht method) and the United Kingdom Function Point User Group (UFPGU, covering the MK11 method). An ISO standard for function point method is also being developed.

Example1: Numerical

Consider a project with the following functional units:

Number of user inputs	= 50
Number of user outputs	= 40
Number of user enquiries	= 35
Number of user files	= 06
Number of external interfaces	= 04

Assume all complexity adjustment factors and weighting factors are average. Compute the function points for the project.

Solution

Solution

We know

$$UFP = \sum_{i=1}^{5} \sum_{J=1}^{3} Z_{ij} w_{ij}$$

UFP = 50 x 4 + 40 x 5 + 35 x 4 + 6 x 10 + 4 x 7

= 200 + 200 + 140 + 60 + 28 = 628

CAF = $(0.65 + 0.01\ \Sigma F_i)$

= (0.65 + 0.01 (14 x 3)) = 0.65 + 0.42 = 1.07

FP = UFP x CAF

= 628 x 1.07 = 672

Example 2:-

- An application has the following:

  10 low external inputs, 12 high external outputs, 20 low internal logical files, 15 high external interface files, 12 average external inquiries, and a value of complexity adjustment factor of 1.10.

  What are the unadjusted and adjusted function point counts ?

Solution

Unadjusted function point counts may be calculated using as:

$$UFP = \sum_{i=1}^{5} \sum_{J=1}^{3} Z_{ij} w_{ij}$$

= 10 x 3 + 12 x 7 + 20 x 7 + 15 + 10 + 12 x 4

= 30 + 84 +140 + 150 + 48

= 452

FP = UFP x CAF

= 452 x 1.10 = 497.2.

FP - Analysis

- Advantages:-
  - Easier to make early estimates.
  - Independent from implementation language, developer experience.

FP - Analysis

- Disadvantages
  - Hard to count automatically
    - Time consuming
  - Complexity factors can be judgmental
    - Repeatability
    - Depends on estimator's experience

Cost Estimation

- For any new software project, it is necessary to know how much will it cost to develop and how much development time will it take.

A number of estimation techniques have been developed and are having following attributes in common.

- Project scope must be established in advance.
- Software metrics are used as a basis from which estimates are made.
- The project is broken into small pieces which are estimated individually.

Cost Estimation

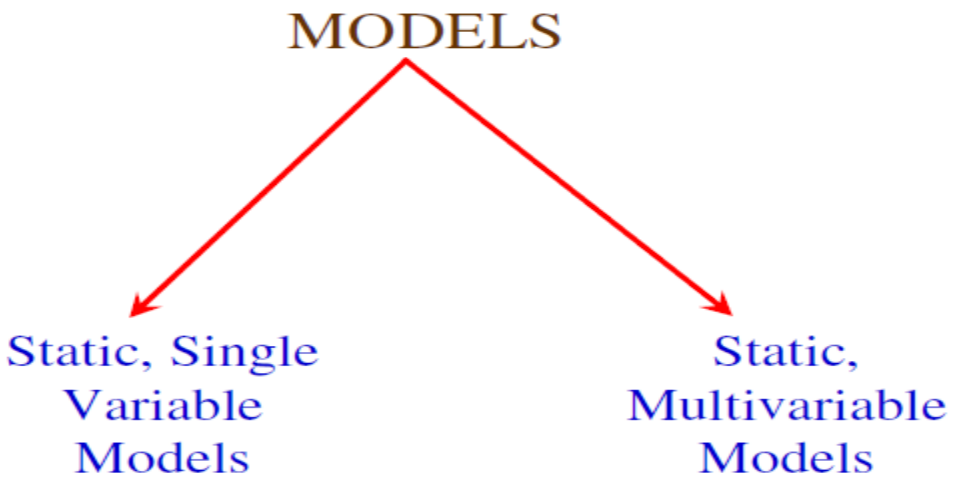
To achieve reliable cost and schedule estimates, a number of option arises:-

- Delay estimation until late in project
- Use simple decomposition techniques to generate cost and time estimates.
- Develop empirical models for estimation.
- Acquire one or more automated estimation tools.

Unfortunately, the first option ,however attractive, is not practical.

Models

The Model is concerned with the representation of the process to be estimated. A model may be static or dynamic.



Models

- In static model a unique variable (Say, size)is used as a key element for calculating others (say cost, time).
- The form of equation used is same for all calculations.
- In dynamic, all variables are interdependent and there is no basic variable as in the static model.
- When a model makes use of a single basic variable to calculate all others it is said to be single-variable model.
- In some models, several variables are needed to describe the software development process, and selected equations combine these variables to give the estimate of cost and development time. These models are called multivariable.

Static, Single Variable Models or SEL model

Methods using this model use an equation to estimate the desired values such as cost, time, effort, etc. They all depend on the same variable used as predictor (say, size). An example of the most common equations is :

SEL

$$C = a\,L^{b} \qquad \text{(i)}$$

C is the cost, L is the size and a,b are constants

$$E = 1.4\,L^{0.93}$$

$$DOC = 30.4\,L^{0.90}$$

$$D = 4.6\,L^{0.26}$$

Effort (E in Person-months), documentation (DOC, in number of pages) and duration (D, in months) are calculated from the number of lines of code (L, in thousands of lines) used as a predictor.

Static, Multivariable Models or Walston and Felix Model

These models are often based on equation (i), they actually depend on several variables representing various aspects of the software development environment, for example method used, user participation, customer oriented changes, memory constraints, etc.

$$E = 5.2\ L^{0.91}$$

$$D = 4.1\ L^{0.36}$$

The productivity index uses 29 variables which are found to be highly correlated to productivity as follows:

$$I = \sum_{i=1}^{29} W_i X_i$$

Where L is Delivered Lines of code in thousands and E is Effort in person-months and D is duration of the development (in months)

W_i is weight factor for ith variable and X_i one of the values –1,0 or +1 depending on whether the variable decreases, has no effect, or increases the productivity respectively.

Example

Example: 4.4

Compare the Walston-Felix model with the SEL model on a software development expected to involve 8 person-years of effort.

(a)Calculate the number of lines of source code that can be produced.

(b)Calculate the duration of the development.

(c)Calculate the productivity in LOC/PY

(d)Calculate the average manning

Solution

Solution

The amount of manpower involved = 8 PY = 96 person-months

(a) Number of lines of source code can be obtained by reversing equation to give:

$$L = (E/a)^{1/b}$$

Then

$$L(SEL) = (96/1.4)^{1/0.93} = 94264 \text{ LOC}$$

$$L(SEL) = (96/5.2)^{1/0.91} = 24632 \text{ LOC.}$$

$E = aL^b$

$L = \left(\frac{E}{a}\right)^{1/b}$

Solution

(b) Duration in months can be calculated by means of equation

$$D(SEL) = 4.6\,(L)^{0.26}$$

$$= 4.6\,(94.264)^{0.26} = 15 \text{ months}$$

$$D(W\text{-}F) = 4.1\,L^{0.36}$$

$$= 4.1(24.632)^{0.36} = 13 \text{ months}$$

(c) Productivity is the lines of code produced per person/month (year)

$$P(SEL) = \frac{94264}{8} = 11783\,LOC\,/\,Person-Years$$

$$P(W-F) = \frac{24632}{8} = 3079\,LOC\,/\,Person-Years$$

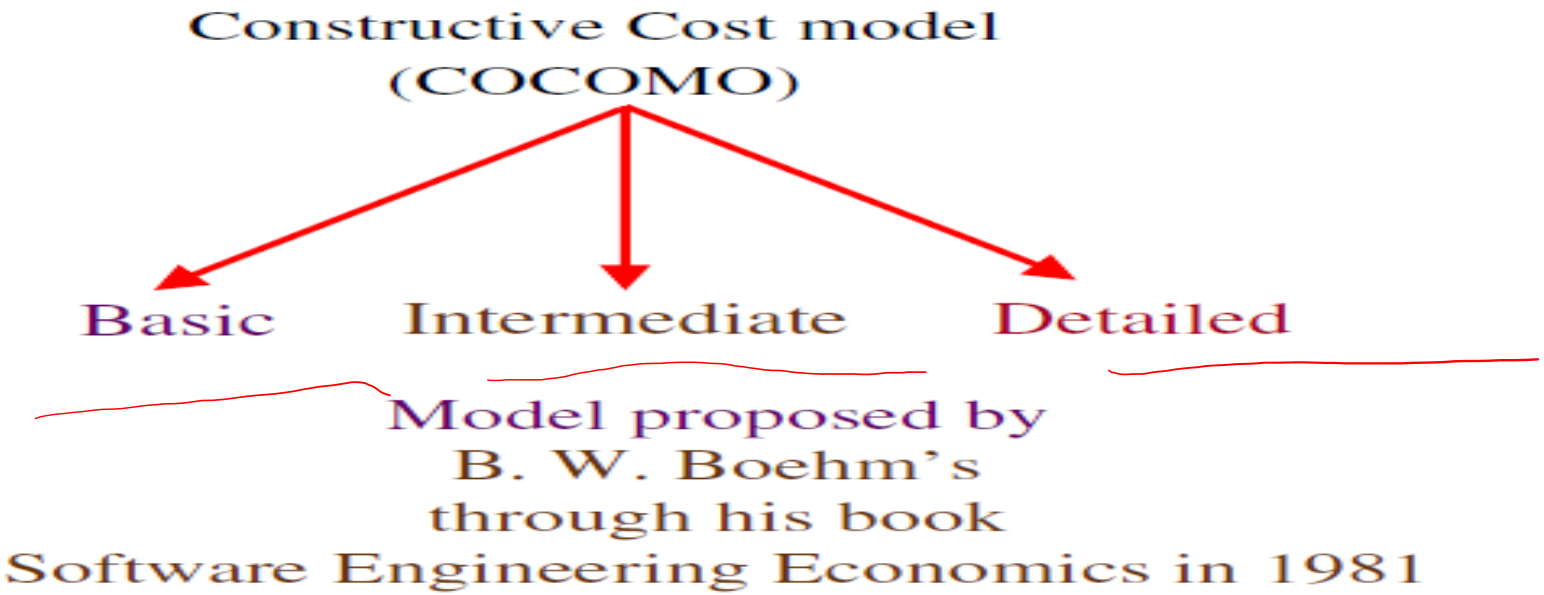
Solution

(d) Average manning is the average number of persons required per month in the project.

$$M(SEL) = \frac{96P - M}{15M} = 6.4 Persons$$

$$M(W - F) = \frac{96P - M}{13M} = 7.4 Persons$$

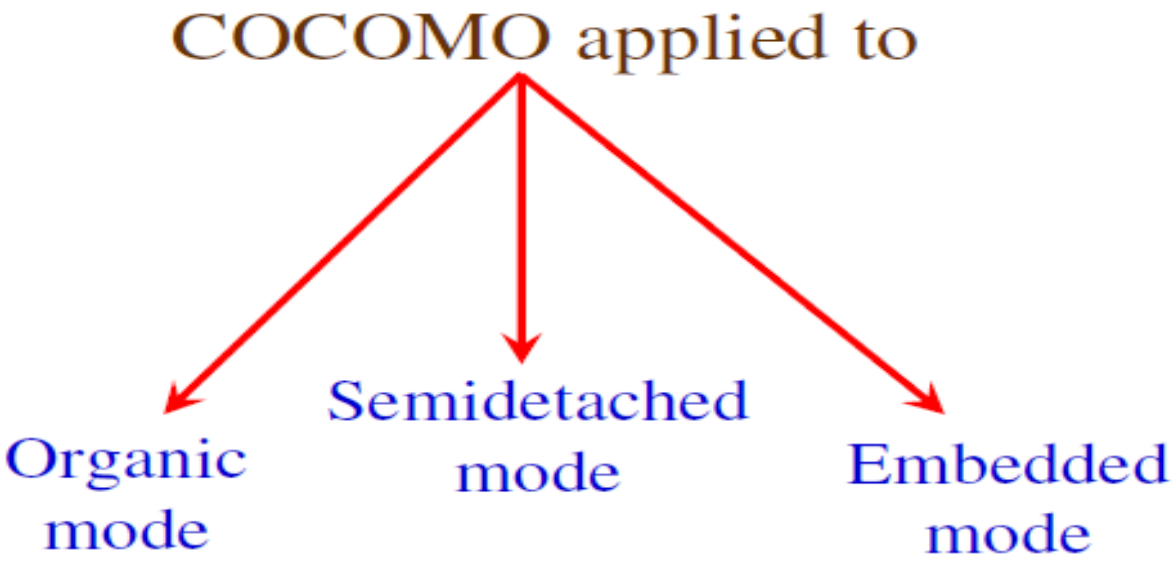
COCOMO -Constructive Cost Model



COCOMO is a hierarchy of software cost estimation models, which include basic, intermediate and detailed sub models.

1. Basic model

This basic model aims at estimating ,in a quick and rough fashion ,most of the small to medium sized software projects.



Comparison of three COCOMO modes

Mode	*Project size*	*Nature of Project*	*Innovation*	*Deadline of the project*	*Development Environment*
Organic	Typically 2-50 KLOC	Small size project, experienced developers in the familiar environment. For example, pay roll, inventory projects etc.	Little	Not tight	Familiar & In house
Semi detached	Typically 50-300 KLOC	Medium size project, Medium size team, Average previous experience on similar project. For example: Utility systems like compilers, database systems, editors etc.	Medium	Medium	Medium
Embedded	Typically over 300 KLOC	Large project, Real time systems, Complex interfaces, Very little previous experience. For example: ATMs, Air Traffic Control etc.	Significant	Tight	Complex Hardware/ customer Interfaces required

Table 4: The comparison of three COCOMO modes

Basic Model

Basic Model

Basic COCOMO model takes the form

$$E = a_b (KLOC)^{b_b}$$

$$D = c_b (E)^{d_b}$$

where E is effort applied in Person-Months, and D is the development time in months. The coefficients a_b, b_b, c_b and d_b are given in table 4 (a).

Basic Model

Software Project	a_b	b_b	c_b	d_b
Organic	2.4	1.05	2.5	0.38
Semidetached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Table 4(a): Basic COCOMO coefficients

Basic Model

When effort and development time are known, the average staff size to complete the project may be calculated as:

$$\text{Average staff size } (SS) = \frac{E}{D} Persons$$

When project size is known, the productivity level may be calculated as:

$$\text{Productivity } (P) = \frac{KLOC}{E} KLOC / PM$$

With the basic model ,the software estimator has a useful tool for estimating quickly ,by two runs on a pocket calculator, the cost and development time of software project, once the size is estimated.

Basic Model Numerical

Example: 4.5

Suppose that a project was estimated to be 400 KLOC. Calculate the effort and development time for each of the three modes i.e., organic, semidetached and embedded.

Solution

The basic COCOMO equation take the form:

$$E = a_b (KLOC)^{b_b}$$

$$D = c_b (KLOC)^{d_b}$$

Estimated size of the project = 400 KLOC

(i) Organic mode

$$E = 2.4(400)^{1.05} = 1295.31 \text{ PM}$$

$$D = 2.5(1295.31)^{0.38} = 38.07 \text{ PM}$$

Continued

(ii) Semidetached mode

$$E = 3.0(400)^{1.12} = 2462.79 \text{ PM}$$

$$D = 2.5(2462.79)^{0.35} = 38.45 \text{ PM}$$

(iii) Embedded mode

$$E = 3.6(400)^{1.20} = 4772.81 \text{ PM}$$

$$D = 2.5(4772.8)^{0.32} = 38 \text{ PM}$$

Basic Model Numerical

Example: 4.6

A project size of 200 KLOC is to be developed. Software development team has average experience on similar type of projects. The project schedule is not very tight. Calculate the effort, development time, average staff size and productivity of the project.

Solution

Solution

The semi-detached mode is the most appropriate mode; keeping in view the size, schedule and experience of the development team.

Hence $E = 3.0(200)^{1.12} = 1133.12$ PM

$D = 2.5(1133.12)^{0.35} = 29.3$ PM

Average staff size $(SS) = \frac{E}{D} Persons$

$$= \frac{1133.12}{29.3} = 38.67 Persons$$

Solution

$$\text{Productivity} = \frac{KLOC}{E} = \frac{200}{1133.12} = 0.1765\, KLOC / PM$$

$$P = 176\ LOC / PM$$

Intermediate Model

- The basic model allowed for a quick and rough estimate, but it resulted in a lack of accuracy. Boehm introduced an additional set of 15 Predictors called cost drivers in the intermediate model to take account of the software development environment.
- Cost drivers are used to adjust the nominal cost of the project to the actual project environment, hence increasing the accuracy of the estimate.

The Cost Drivers are grouped into four categories:-

1. Product attributes
   a) Required software reliability (RELY)
   b) Database size (DATA)
   c) Product complexity (CPLX)
2. Computer attributes
   a) Execution time constraint (TIME)
   b) Main storage constraint (STOR)
   c) Virtual machine volatility (VIRT)
   d) Computer turnaround time (TURN)

Cost Drivers

3. Personnel attributes
   a) Analyst capability (ACAP)
   b) Application experience (AEXP)
   c) Programmer capability (PCAP)
   d) Virtual machine experience (VEXP)
   e) Programming language experience (LEXP)
4. Project attributes
   a) Modern programming practices (MODP)
   b) Use of software tools (TOOL)
   c) Required development schedule (SCED)

Cost drivers

- Each cost driver is rated for a given project environment.
- The rating uses a scale very low,low,nomial,high ,very high, extra high which describes to what extent the cost driver applies to the project being estimated.

Intermediate Model

Multipliers of different cost drivers

Cost Drivers	RATINGS					
	Very low	Low	Nominal	High	Very high	Extra high
Product Attributes						
RELY	0.75	0.88	1.00	1.15	1.40	--
DATA	--	0.94	1.00	1.08	1.16	--
CPLX	0.70	0.85	1.00	1.15	1.30	1.65
Computer Attributes						
TIME	--	--	1.00	1.11	1.30	1.66
STOR	--	--	1.00	1.06	1.21	1.56
VIRT	--	0.87	1.00	1.15	1.30	--
TURN	--	0.87	1.00	1.07	1.15	--

Intermediate Model

Cost Drivers	RATINGS					
	Very low	Low	Nominal	High	Very high	Extra high
Personnel Attributes						
ACAP	1.46	1.19	1.00	0.86	0.71	--
AEXP	1.29	1.13	1.00	0.91	0.82	--
PCAP	1.42	1.17	1.00	0.86	0.70	--
VEXP	1.21	1.10	1.00	0.90	--	--
LEXP	1.14	1.07	1.00	0.95	--	--
Project Attributes						
MODP	1.24	1.10	1.00	0.91	0.82	--
TOOL	1.24	1.10	1.00	0.91	0.83	--
SCED	1.23	1.08	1.00	1.04	1.10	--

Table 5: Multiplier values for effort calculations

Intermediate Model

Intermediate COCOMO equations

$$E = a_i (KLOC)^{b_i} * EAF$$

$$D = c_i (E)^{d_i}$$

Project	a_i	b_i	c_i	d_i
Organic	3.2	1.05	2.5	0.38
Semidetached	3.0	1.12	2.5	0.35
Embedded	2.8	1.20	2.5	0.32

Numerical on Intermediate Model

- A new project with estimated 400 KLOC embedded system has to be developed. Project manager has a choice of hiring from two pools of developers: Very highly capable with very little experience in the programming language being used

OR Developers of low quality but a lot of experience with the programming language. What is the impact of hiring all developers from one or the other pool ?

Solution

This is the case of embedded mode and model is intermediate COCOMO.

Hence $$E = a_i (KLOC)^{d_i}$$

$$= 2.8\ (400)^{1.20} = 3712 \text{ PM}$$

Case I: Developers are very highly capable with very little experience in the programming being used.

EAF $= 0.82 \times 1.14 = 0.9348$

E $= 3712 \times .9348 = 3470$ PM

D $= 2.5\ (3470)^{0.32} = 33.9$ M

Case II: Developers are of low quality but lot of experience with the programming language being used.

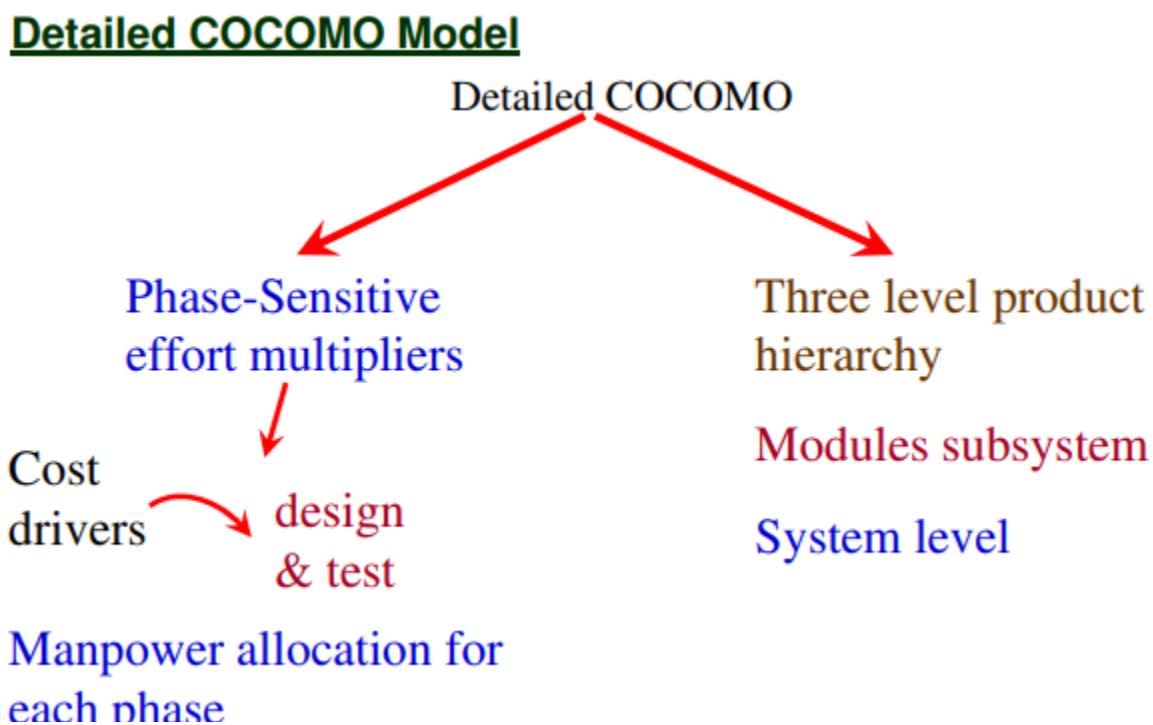
$$EAF = 1.29 \times 0.95 = 1.22$$

$$E = 3712 \times 1.22 = 4528 \text{ PM}$$

$$D = 2.5\ (4528)^{0.32} = 36.9 \text{ M}$$

Case II requires more effort and time. Hence, low quality developers with lot of programming language experience could not match with the performance of very highly capable developers with very litter experience.

Detailed COCOMO



Detailed COCOMO

Development Phase

Plan / Requirements

EFFORT	:	6% to 8%
DEVELOPMENT TIME	:	10% to 40%

% depend on mode & size

Detailed COCOMO

Design

Effort	:	16% to 18%
Time	:	19% to 38%

Programming

Effort	:	48% to 68%
Time	:	24% to 64%

Integration & Test

Effort	:	16% to 34%
Time	:	18% to 34%

Detailed COCOMO

- As the software might be partly developed from software already existing (that is, re-usable code), a full development is not always required. In such cases, the parts of design document (DD%), code (C%) and integration (I%) to be modified are estimated. Then, an adjustment factor, A, is calculated by means of the following equation.

A = 0.4 DD + 0.3 C + 0.3 I

The size equivalent is obtained by

S (equivalent) = (S x A) / 100

$Ep = mp\ E$

$Dp = t\ p\ D$

Lifecycle Phase Values of μ_p

Mode & Code Size	Plan & Requirements	System Design	Detailed Design	Module Code & Test	Integration & Test
Organic Small S≈2	0.06	0.16	0.26	0.42	0.16
Organic medium S≈32	0.06	0.16	0.24	0.38	0.22
Semidetached medium S≈32	0.07	0.17	0.25	0.33	0.25
Semidetached large S≈128	0.07	0.17	0.24	0.31	0.28
Embedded large S≈128	0.08	0.18	0.25	0.26	0.31
Embedded extra large S≈320	0.08	0.18	0.24	0.24	0.34

Lifecycle Phase Values of τ_p

Mode & Code Size	Plan & Requirements	System Design	Detailed Design	Module Code & Test	Integration & Test
Organic Small S≈2	0.10	0.19	0.24	0.39	0.18
Organic medium S≈32	0.12	0.19	0.21	0.34	0.26
Semidetached medium S≈32	0.20	0.26	0.21	0.27	0.26
Semidetached large S≈128	0.22	0.27	0.19	0.25	0.29
Embedded large S≈128	0.36	0.36	0.18	0.18	0.28
Embedded extra large S≈320	0.40	0.38	0.16	0.16	0.30

What is UML?

- UML stands for “Unified Modeling Language”
- Modeling is describing system at a high level of abstraction
- It is a industry-standard graphical language for specifying, visualizing, constructing, and documenting the artifacts of software systems
- The UML uses mostly graphical notations to express the OO analysis and design of software projects.
- Simplifies the complex process of software design

Why UML for Modeling/Benefits of UML

- Use graphical notation to communicate more clearly than natural language (imprecise) and code(too detailed).
- Quick understanding
- Help acquire an overall view of a system.
- Lessens chances of conflicts.
- UML is not dependent on any one language or technology.
- Necessary to manage complexity.
- UML moves us from fragmentation to standardization.

Types of UML Diagrams

- **Use Case Diagram**
- **Class Diagram**
- **Activity Diagram**
- **Sequence Diagram**
- **Collaboration Diagram**
- **State Diagram**
- **Deployment Diagram**

Use Case Diagram

- Used for describing a set of user **scenarios**
- Illustrates functionality provided by the system
- Mainly used for capturing user requirements
- Work like a **contract** between end user and software developers

Use Case Diagram (core components)

Actors: A role that a user plays with respect to the system,including human users and other systems. e.g.,inanimate physical objects (e.g. robot); an external system that needs some information from the current system.

Use case: A set of scenarios that describing an interaction between a user and a system, including alternatives.

Actor

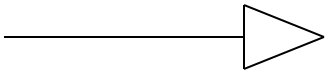
Use Case

System boundary: rectangle diagram representing the boundary between the actors and the system.

Use Case Diagram(core relationship)

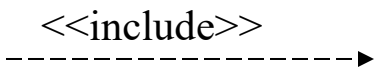
Association: communication between an actor and a use case; Represented by a solid line.

Generalization: relationship between one general use case and a special use case (used for defining special alternatives)
Represented by a line with a triangular arrow head toward the parent use case.

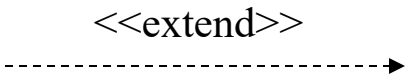


Use Case Diagram(core relationship)

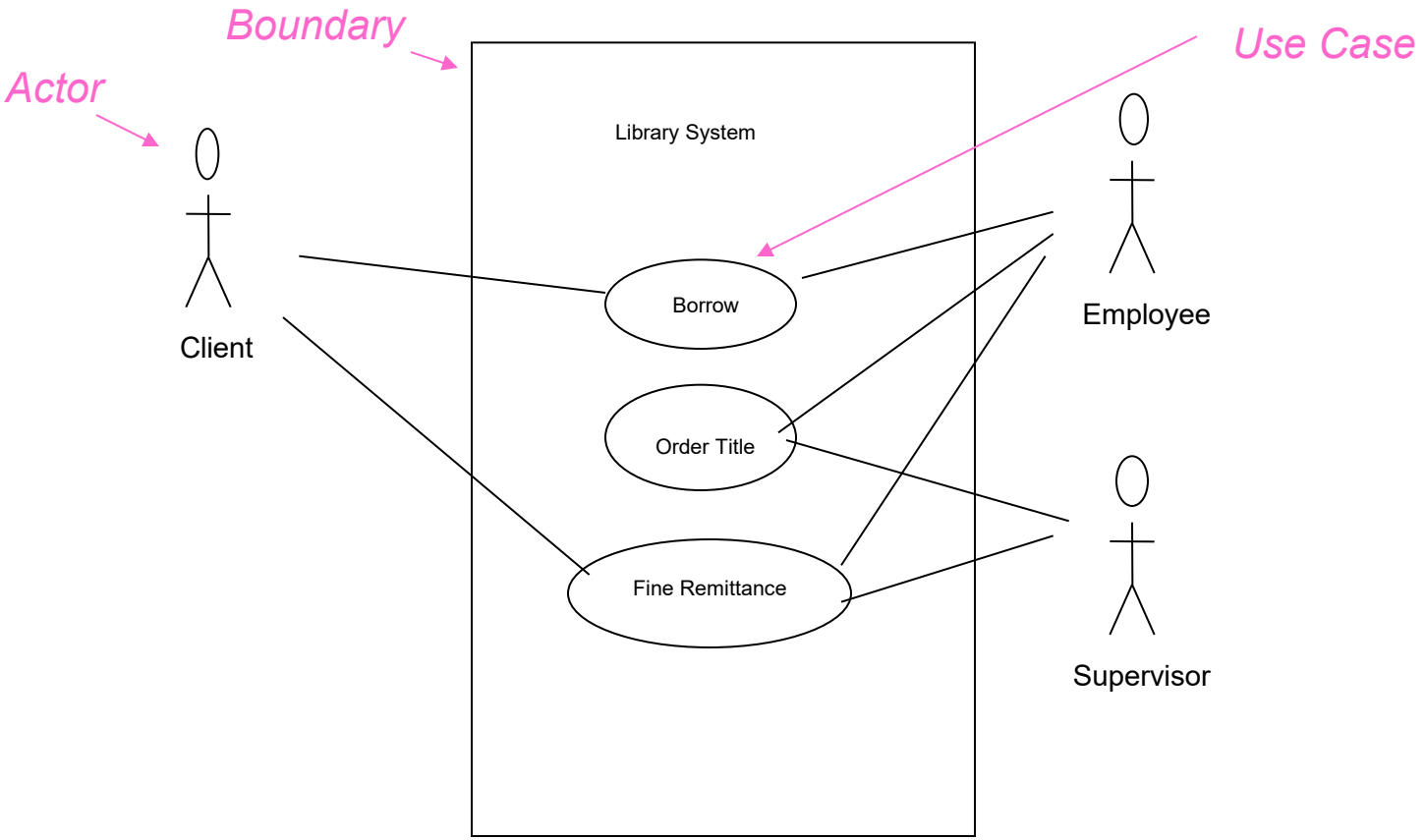
<u>Include</u>: a dotted line labeled <<include>> beginning at base use case and ending with an arrows pointing to the include use case. The include relationship occurs when a chunk of behavior is similar across more than one use case. Use "include" in stead of copying the description of that behavior.



<u>Extend</u>: a dotted line labeled <<extend>> with an arrow toward the base case. The extending use case may add behavior to the base use case. The base class declares "extension points".

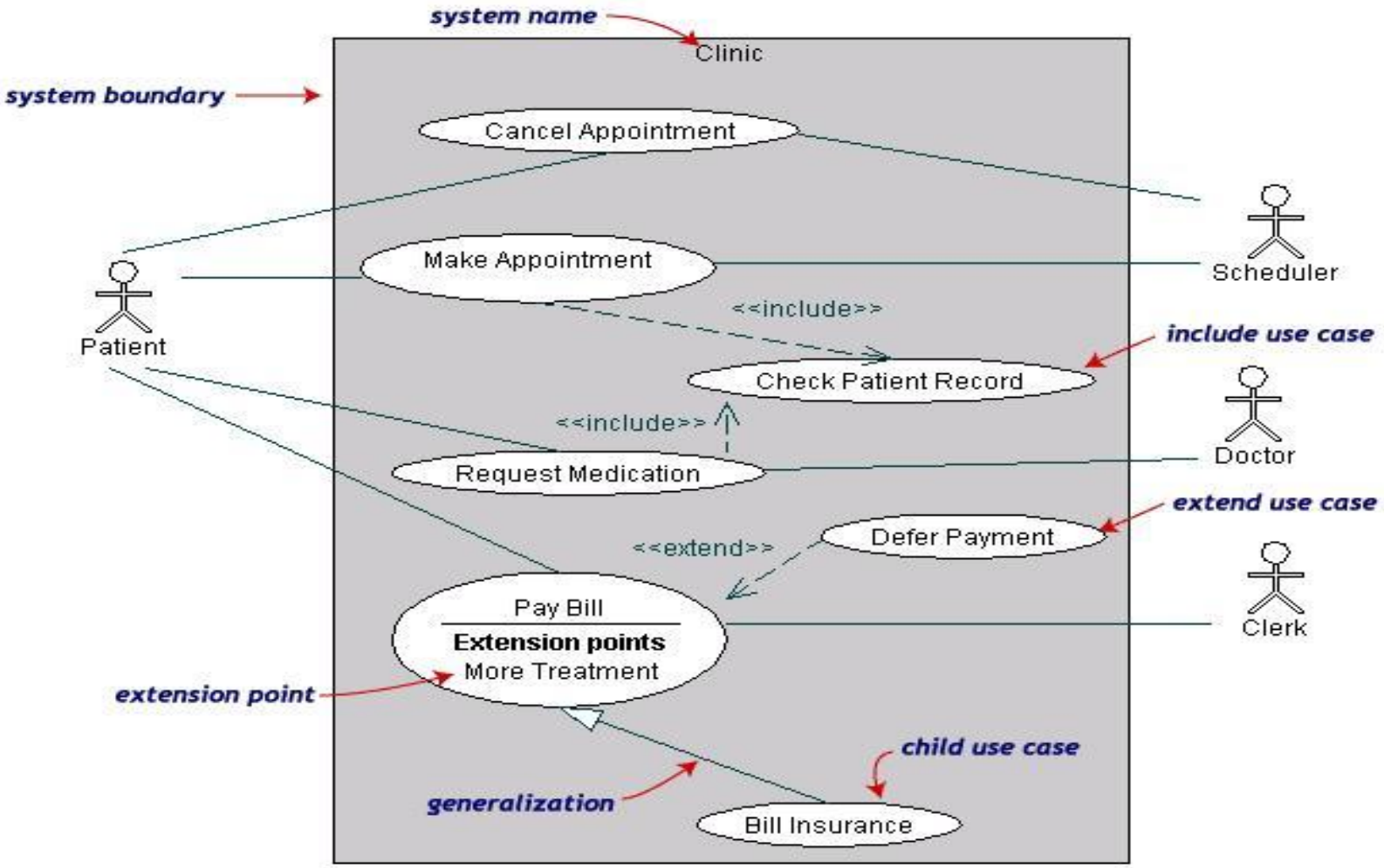


Use Case Diagrams



- A generalized description of how a system will be used.
- Provides an overview of the intended functionality of the system

Use Case Diagrams(cont.)



Use Case Diagrams(cont.)

- **Pay Bill** is a parent use case and **Bill Insurance** is the child use case. (generalization)

- Both **Make Appointment** and **Request Medication** include **Check Patient Record** as a subtask.(include)

- The **extension point** is written inside the base case **Pay bill**; the extending class **Defer payment** adds the behavior of this extension point. (extend)

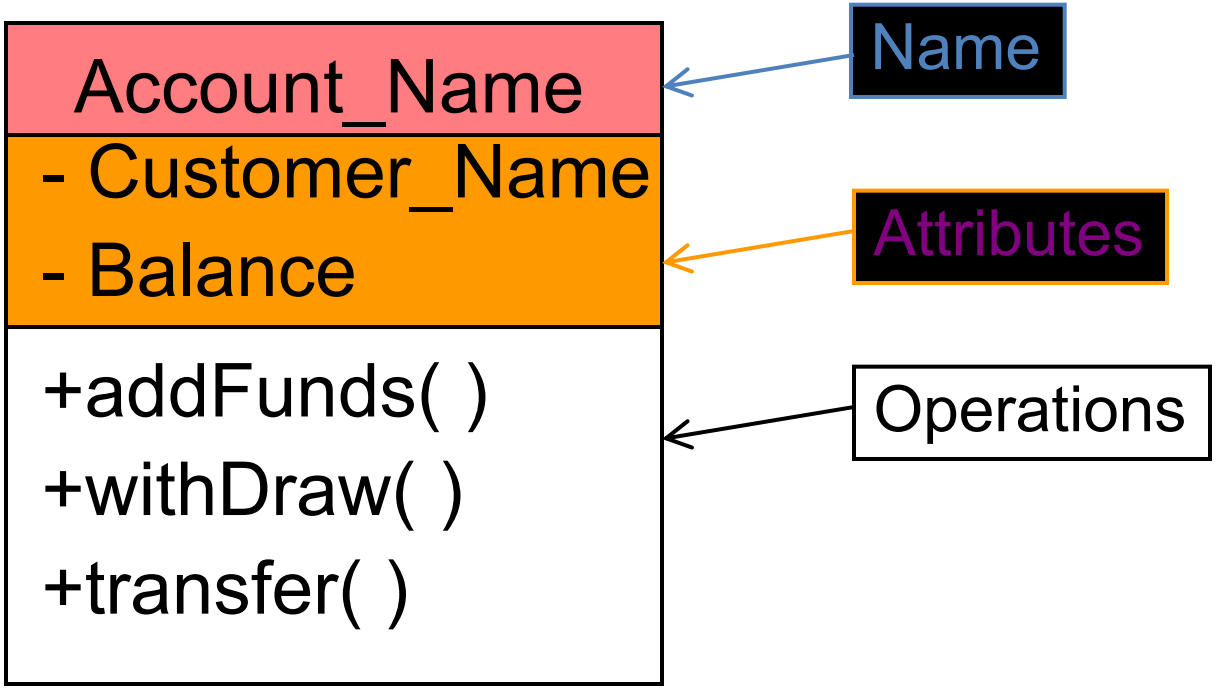
Class diagram

- Used for describing structure and behavior in the use cases
- Provide a conceptual model of the system in terms of entities and their relationships
- Used for requirement capture, end-user interaction
- Detailed class diagrams are used for developers

Class representation

- Each class is represented by a rectangle subdivided into three compartments
  - Name
  - Attributes
  - Operations
- Modifiers are used to indicate visibility of attributes and operations.
  - '+' is used to denote *Public* visibility (everyone)
  - '#' is used to denote *Protected* visibility (friends and derived)
  - '-' is used to denote *Private* visibility (no one)
- By default, attributes are hidden and operations are visible.

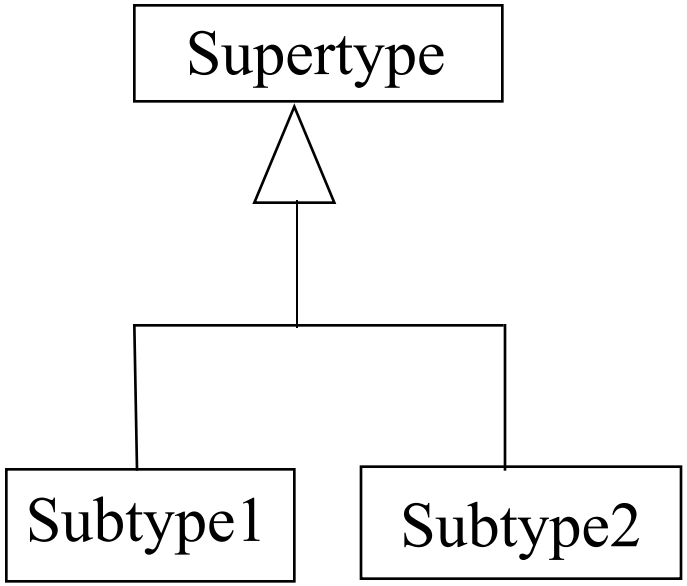
An example of Class



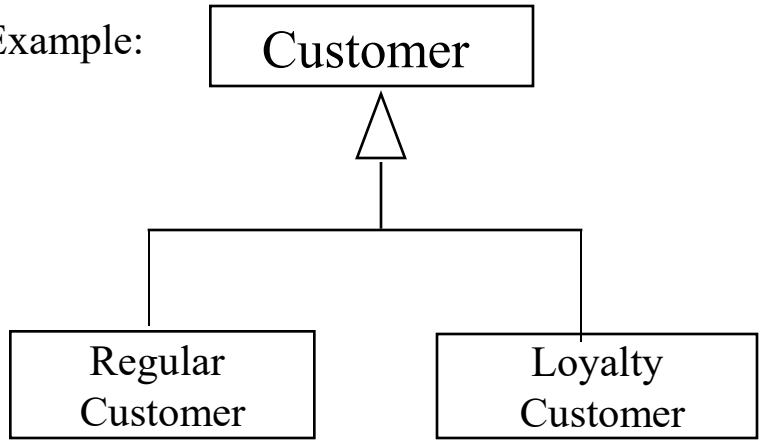
OO Relationships

- There are two kinds of Relationships
  - Generalization (parent-child relationship)
  - Association (student enrolls in course)
- Associations can be further classified as
  - Aggregation
  - Composition

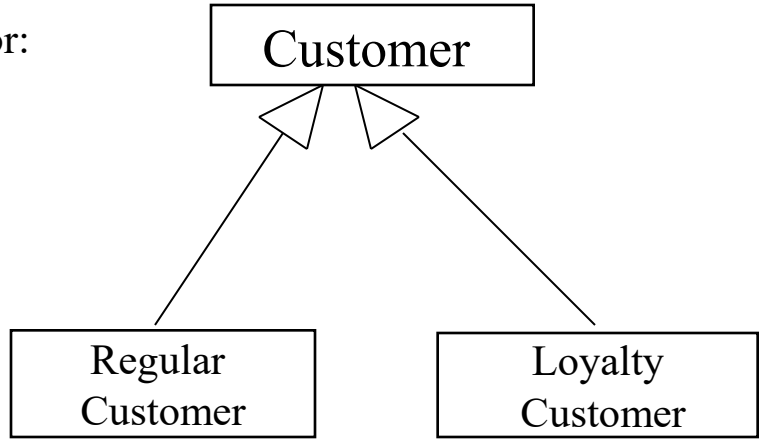
OO Relationships: **Generalization**



Example:



or:



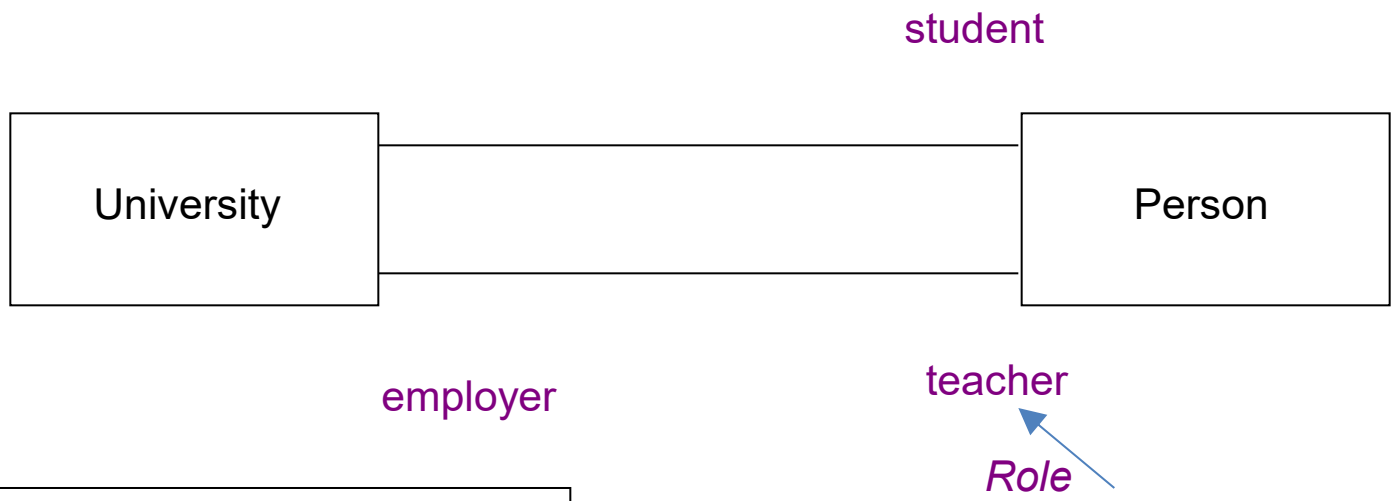
- Generalization expresses a parent/child relationship among related classes.

- Used for abstracting details in several layers

OO Relationships: **Association**

- Represent relationship between instances of classes
  - Student enrolls in a course
  - Courses have students
  - Courses have exams
  - Etc.
- Association has two ends
  - Role names (e.g. enrolls)
  - Multiplicity (e.g. One course can have many students)
  - Navigability (unidirectional, bidirectional)

Association: Multiplicity and Roles



Multiplicity

Symbol	Meaning
1	One and only one
0..1	Zero or one
M..N	From M to N (natural language)
*	From zero to any positive integer
0..*	From zero to any positive integer
1..*	From one to any positive integer

Role

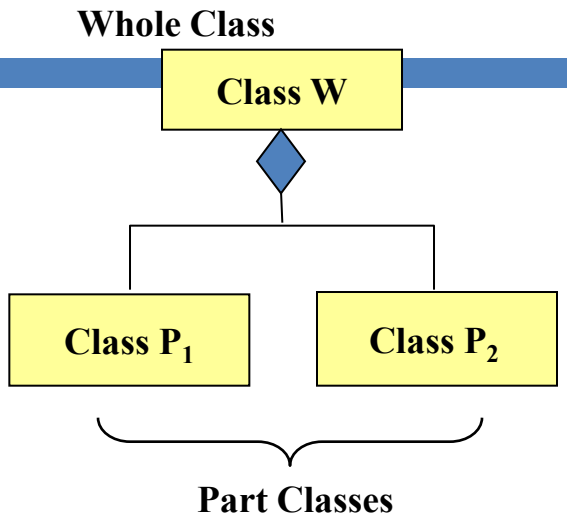
"A given university groups many people; some act as students, others as teachers. A given student belongs to a single university; a given teacher may or may not be working for the university at a particular time."

Association: Model to Implementation

```
Class Student {
    Course enrolls[4];
}

Class Course {
    Student have[];
}
```

OO Relationships: **Composition**



Composition: expresses a relationship among instances of related classes. It is a specific kind of Whole-Part relationship.

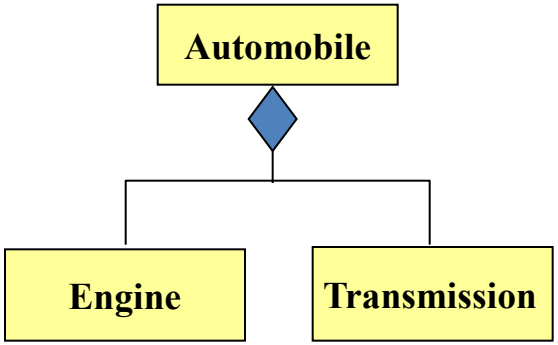
It expresses a relationship where an instance of the Whole-class has the responsibility to create and initialize instances of each Part-class.

It may also be used to express a relationship where instances of the Part-classes have privileged access or visibility to certain attributes and/or behaviors defined by the Whole-class.

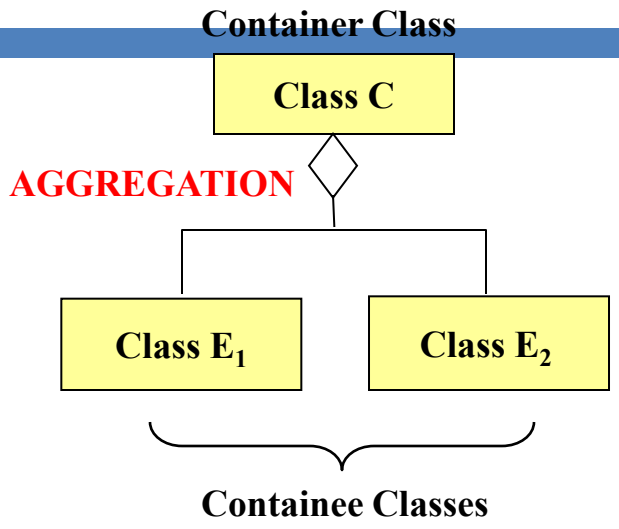
Composition should also be used to express relationship where instances of the Whole-class have exclusive access to and control of instances of the Part-classes.

Composition should be used to express a relationship where the behavior of Part instances is undefined without being related to an instance of the Whole. And, conversely, the behavior of the Whole is ill-defined or incomplete if one or more of the Part instances are undefined.

Example



OO Relationships: **Aggregation**



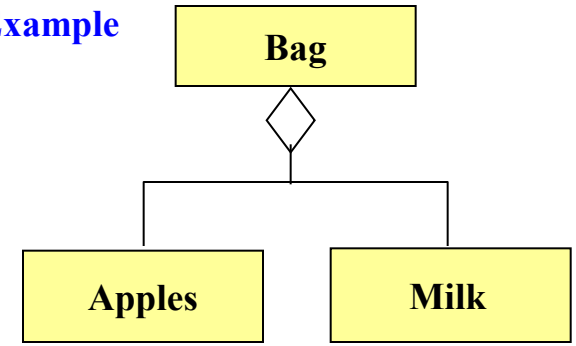
Aggregation: expresses a relationship among instances of related classes. It is a specific **kind of Container-Containee** relationship.

It expresses a relationship where an instance of the Container-class has the responsibility to **hold and maintain instances** of each Containee-class that have been created outside the auspices of the Container-class.

Aggregation should be used to express a more informal relationship than composition expresses. That is, it is an appropriate relationship where the **Container and its Containees**

Aggregation is appropriate when **Container and Containees** have no special access privileges to each other.

Example



Aggregation vs. Composition

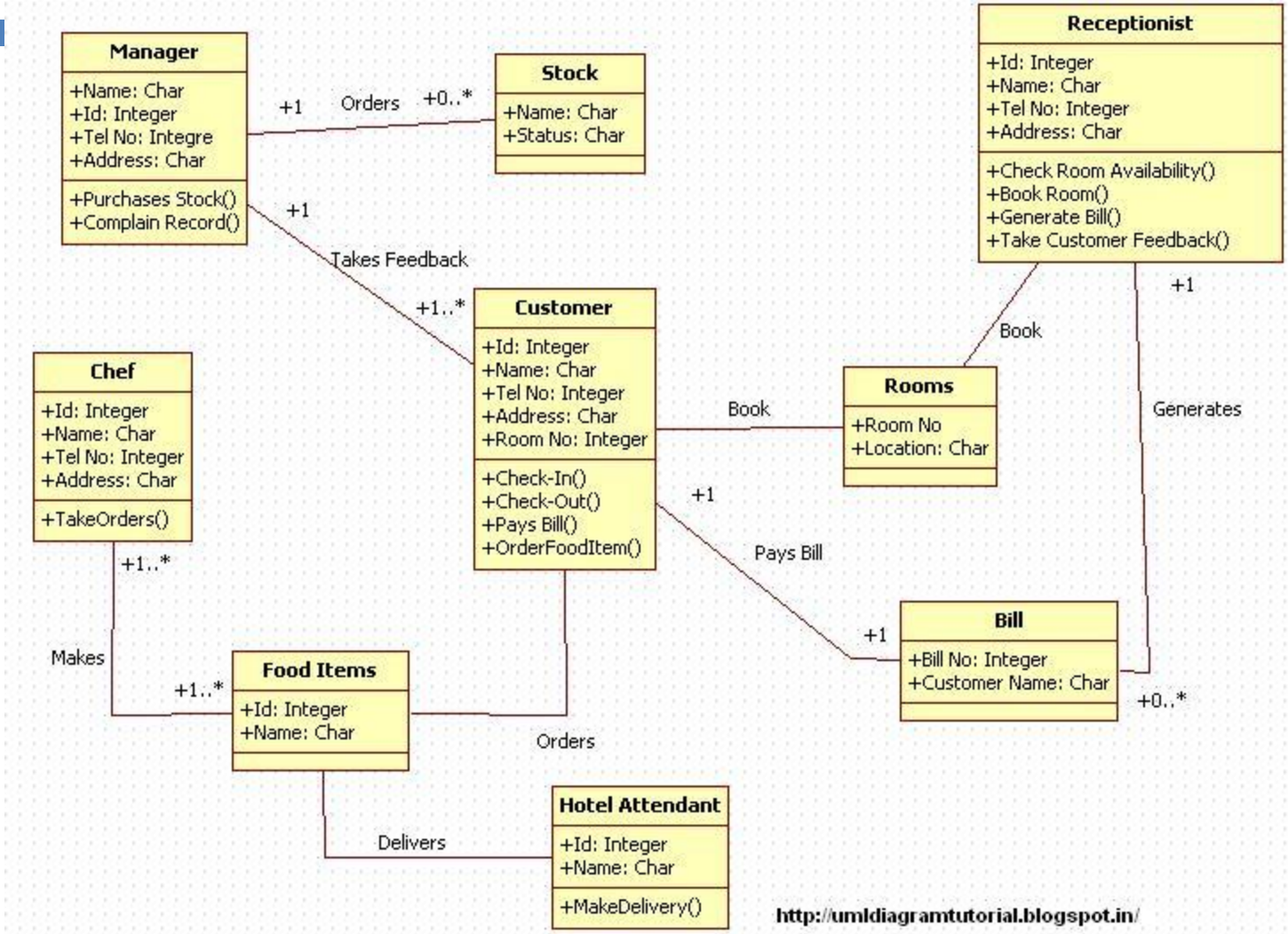
- **Composition** is really a strong form of **aggregation**
  - components have only one owner
  - components cannot exist independent of their owner
  - components live or die with their owner

  e.g. Each car has an engine that can not be shared with other cars.

- **Aggregations** may form "part of" the aggregate, but may not be essential to it. They may also exist independent of the aggregate.

  e.g. Apples may exist independent of the bag.

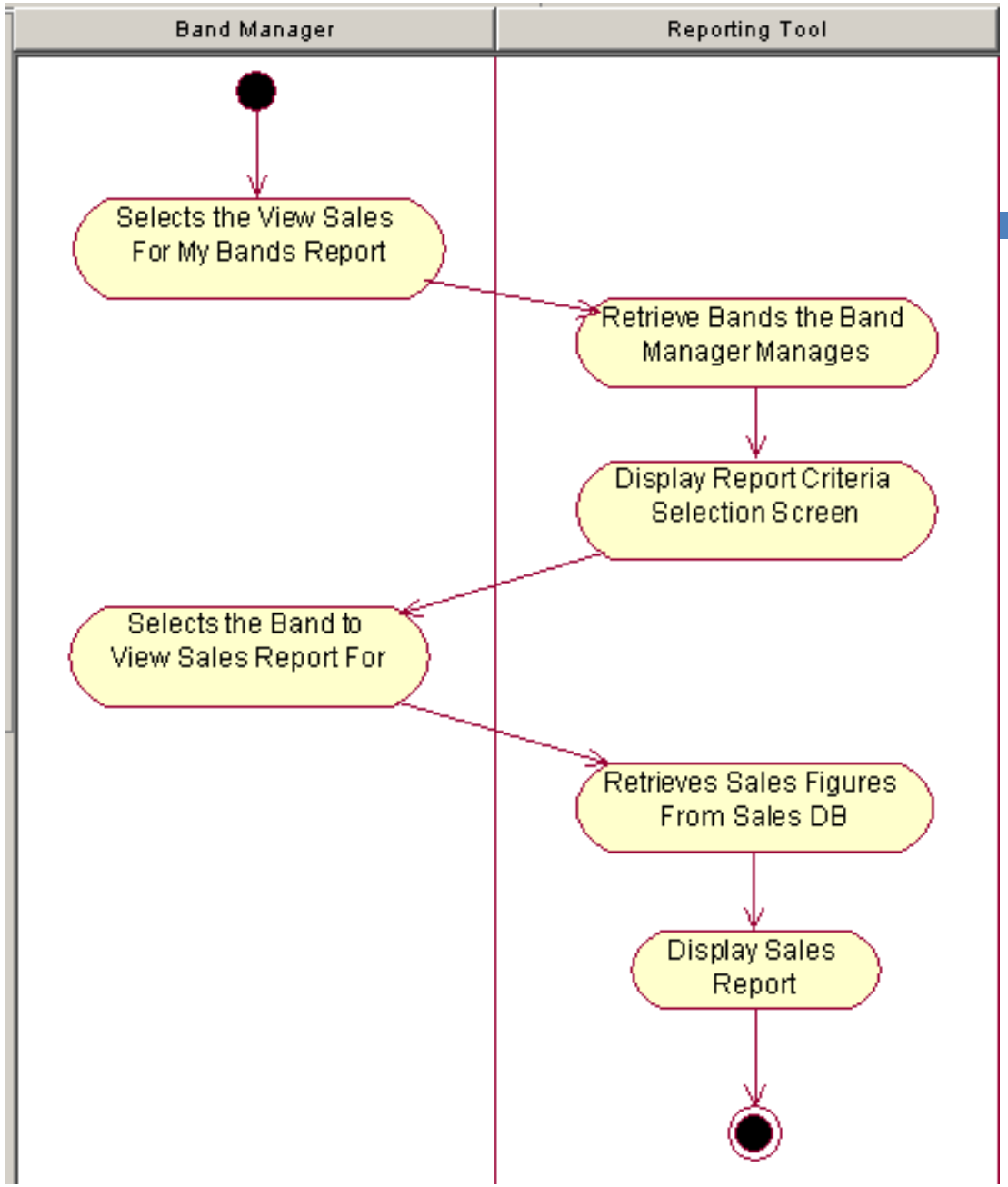
Class Diagram Example



Hotel Management System

Activity Diagram

- Procedural flow of control between two or more class objects
- Activities are grouped into swim lanes.
- Activity is modeled by drawing a rectangle with rounded edges, enclosing the activity's name.
- Connected to other activities through transition lines
- Activities that terminate the modeled process are connected to a termination point

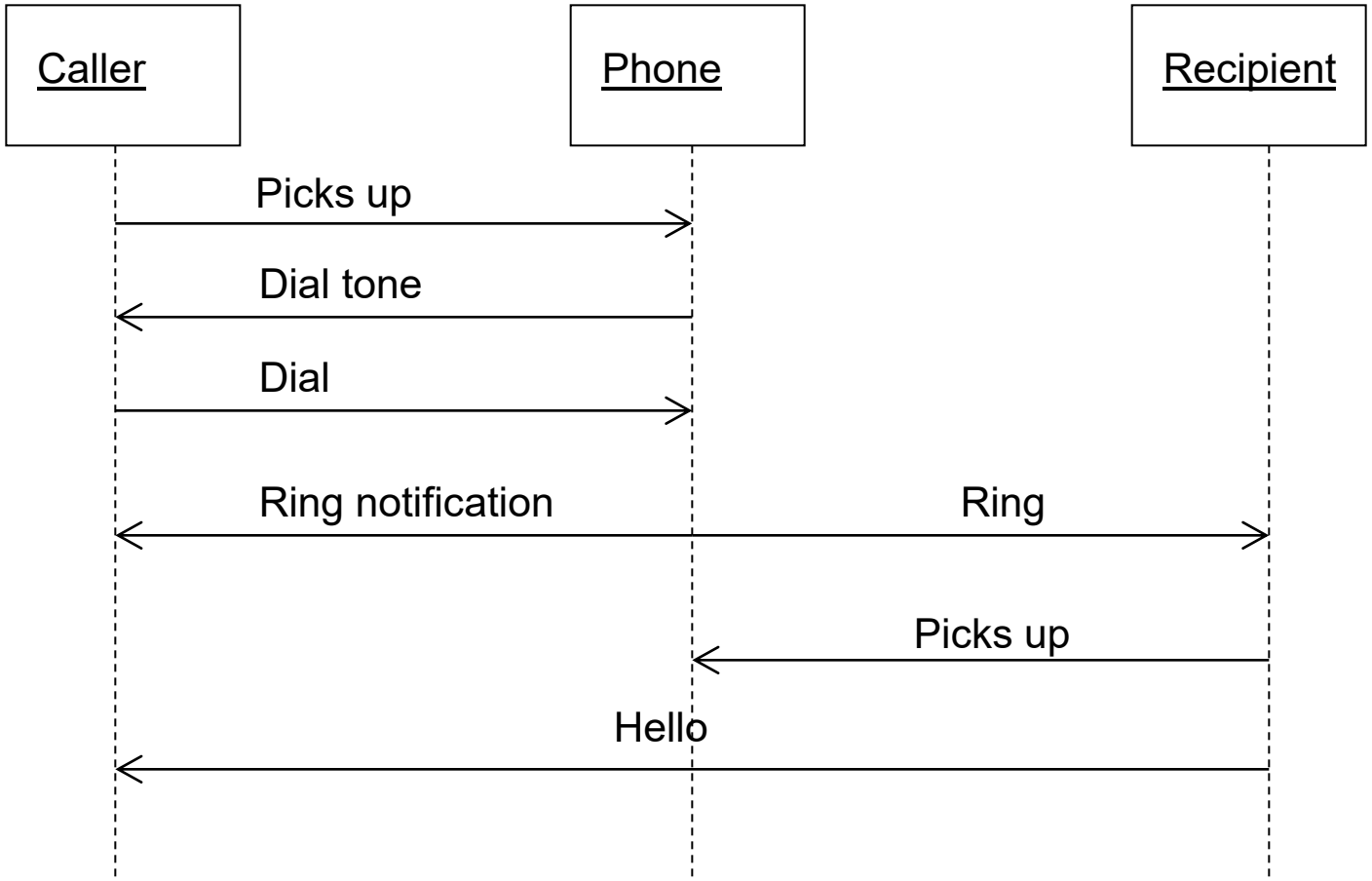


Activity diagram, with two swim lanes to indicate control of activity by two objects: the band manager, and the reporting tool

Sequence Diagram

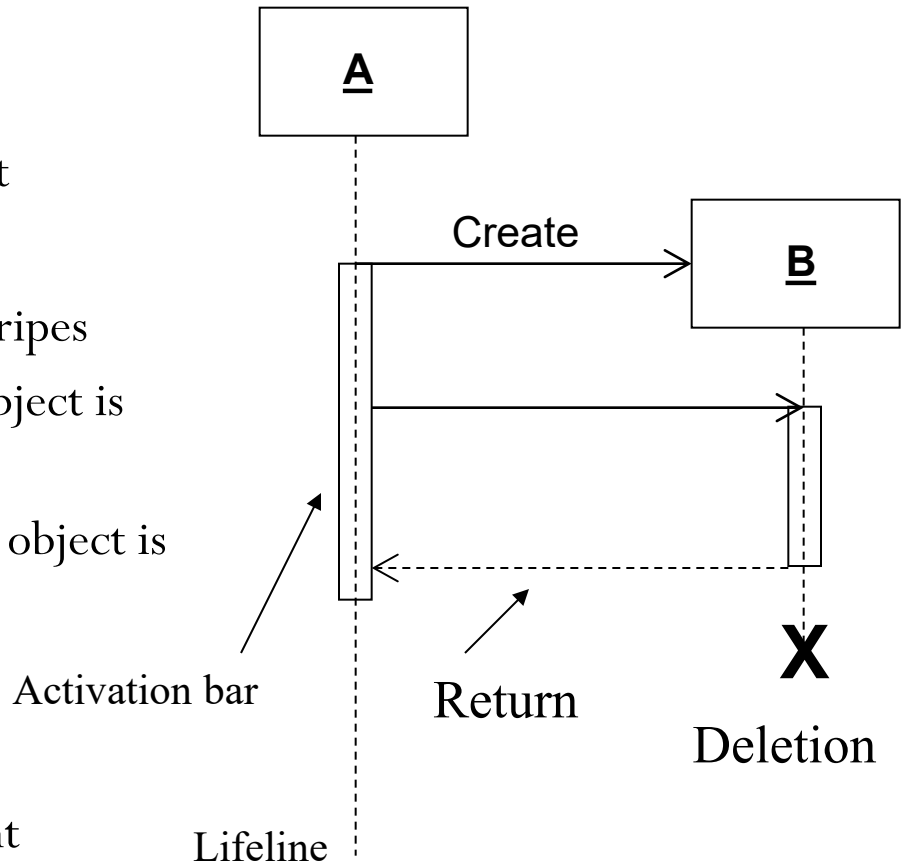
- Two dimensional
- Vertical dimension shows sequence of messages in the time order that they occur.
- Horizontal dimension shows the object instances to which the messages are sent.
- Self-Call can also be there in which a message that an object sends to itself.

Sequence Diagram(make a phone call)

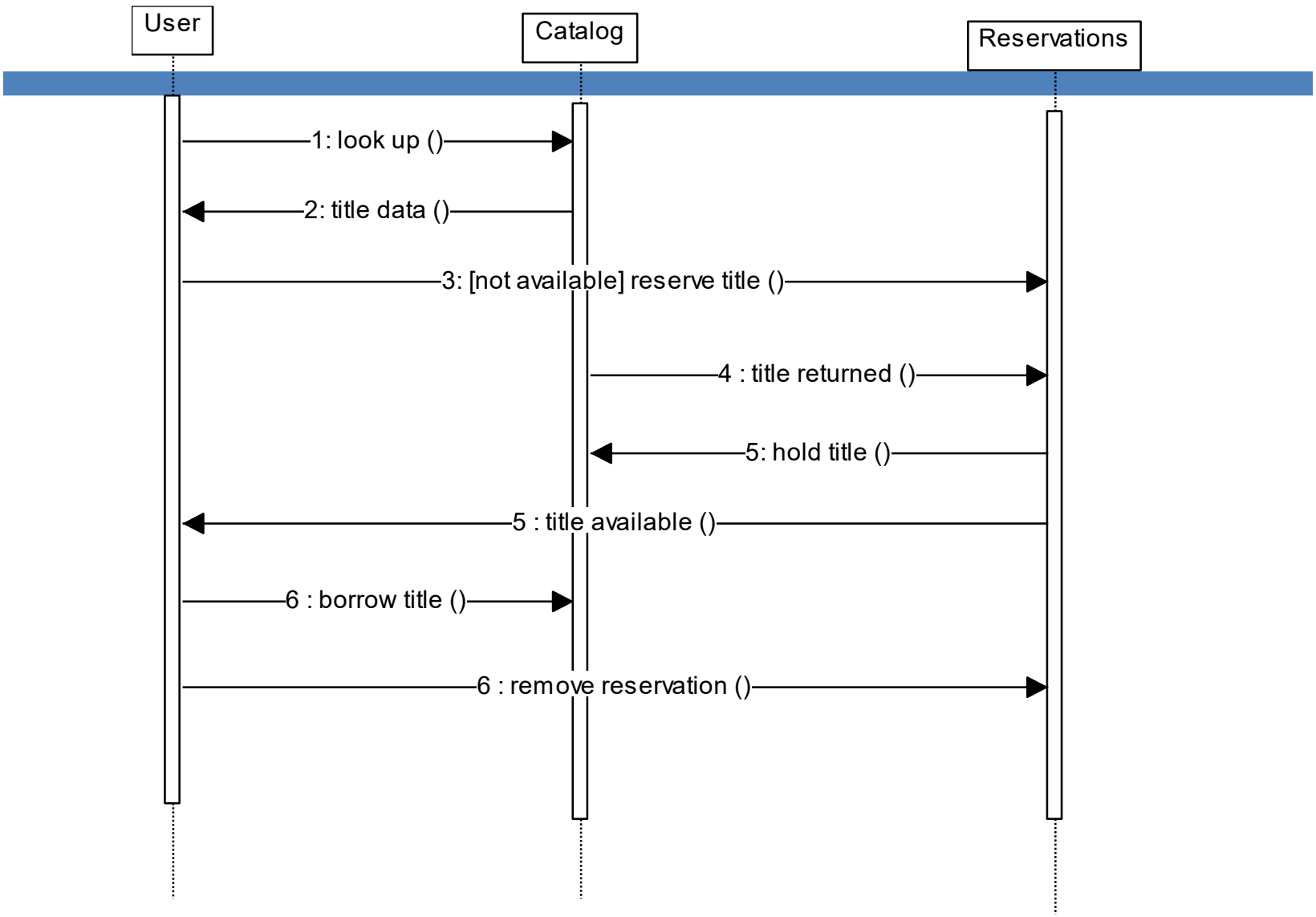


Sequence Diagrams – Object Life Spans

- Creation
  - Create message
  - Object life starts at that point
- Activation
  - Symbolized by rectangular stripes
  - Place on the lifeline where object is activated.
  - Rectangle also denotes when object is deactivated.
- Deletion
  - Placing an 'X' on lifeline
  - Object's life ends at that point



Sequence Diagram

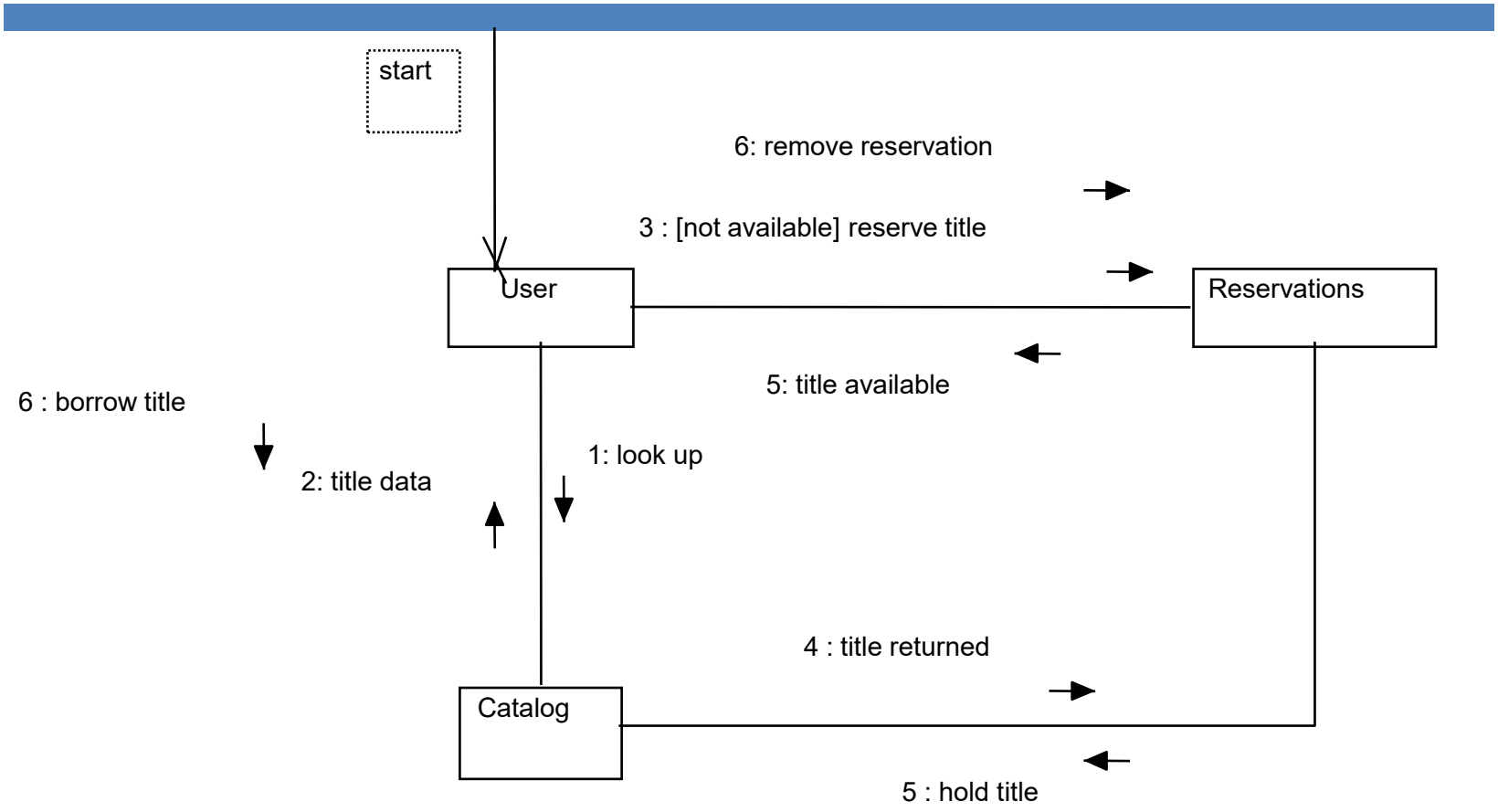


Issuing a book from library

Collaboration Diagram

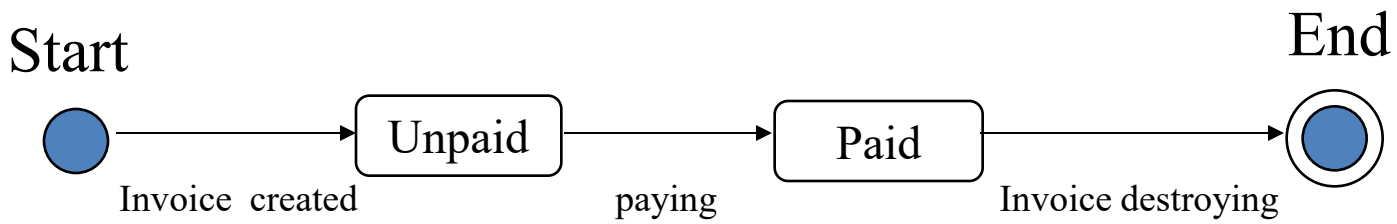
- Shows the relationship between objects and the order of messages passed between them.
- The objects are listed as rectangles and arrows indicate the messages being passed
- Convey the same information as sequence diagrams, but focus on object roles instead of the time sequence.

Collaboration diagrams of earlier made sequence diagram of issuing of book from library



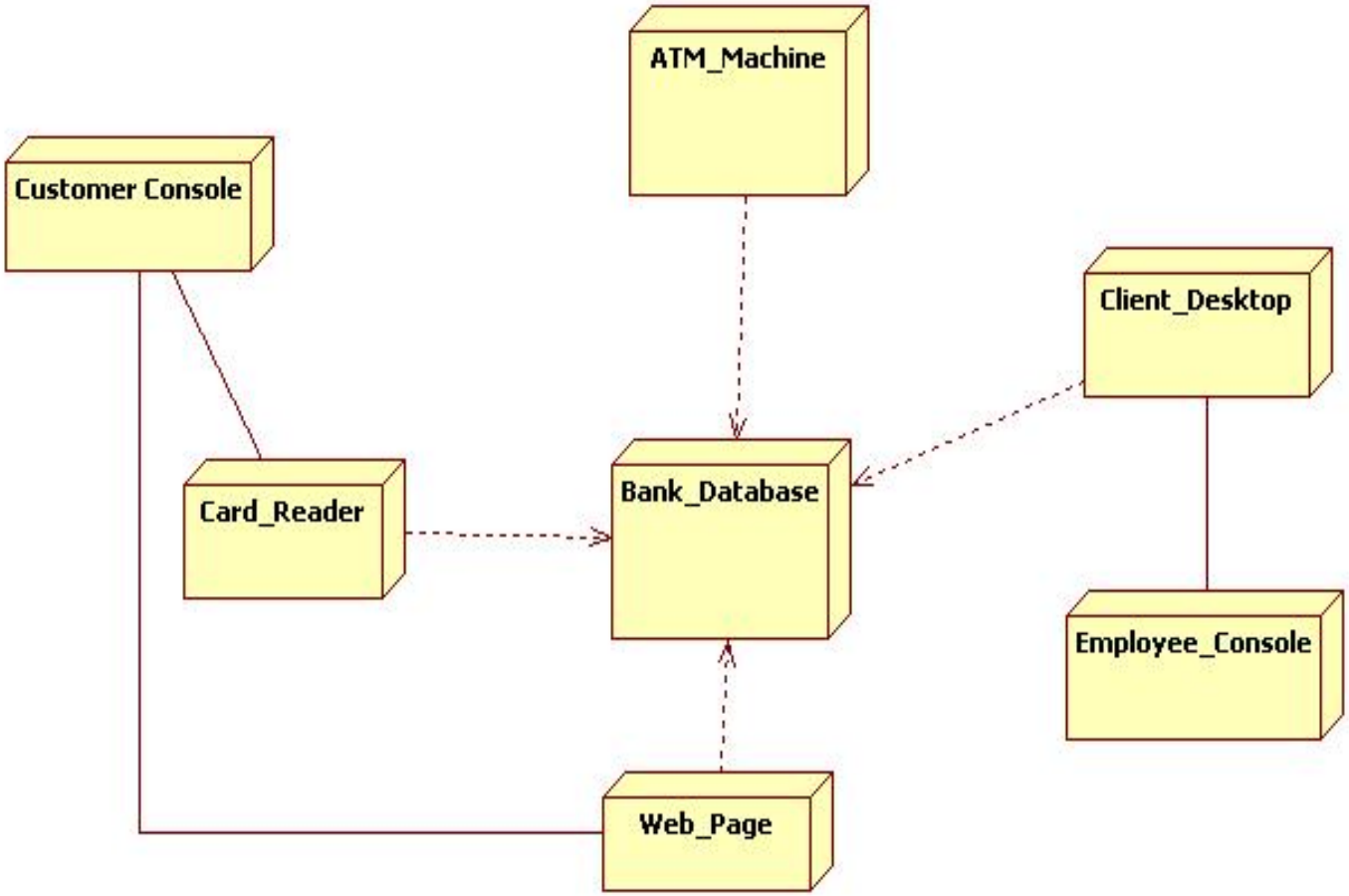
State Diagrams

State Diagrams show the sequences of states an object goes through during its life cycle in response to stimuli, together with its responses and actions; an abstraction of all possible behaviors.



(Billing Example)

Deployment Diagram

- System will be physically deployed in the hardware environment
- System's production staff makes its considerable use
- Purpose is to show where the different components of the system will physically run and how they will communicate with each other

Deployment for ATM Machine